



Data cleaning and machine learning: a systematic literature review

Pierre-Olivier Côté¹ · Amin Nikanjam¹ · Nafisa Ahmed¹ · Dmytro Humeniuk¹ · Foutse Khomh¹

Received: 3 October 2023 / Accepted: 29 May 2024 / Published online: 11 June 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

Machine Learning (ML) is integrated into a growing number of systems for various applications. Because the performance of an ML model is highly dependent on the quality of the data it has been trained on, there is a growing interest in approaches to detect and repair data errors (i.e., data cleaning). Researchers are also exploring how ML can be used for data cleaning; hence creating a dual relationship between ML and data cleaning. To the best of our knowledge, there is no study that comprehensively reviews this relationship. This paper's objectives are twofold. First, it aims to summarize the latest approaches for data cleaning for ML and ML for data cleaning. Second, it provides future work recommendations. We conduct a systematic literature review of the papers published between 2016 and 2022 inclusively. We identify different types of data cleaning activities with and for ML: feature cleaning, label cleaning, entity matching, outlier detection, imputation, and holistic data cleaning. We summarize the content of 101 papers covering various data cleaning activities and provide 24 future work recommendations. Our review highlights many promising data cleaning techniques that can be further extended. We believe that our review of the literature will help the community develop better approaches to clean data.

Keywords Machine learning · Data cleaning · Systematic literature review · Survey · Taxonomy

This work is funded by the Fonds de Recherche du Quebec (FRQ), the Canadian Institute for Advanced Research (CIFAR), and the Natural Sciences and Engineering Research Council of Canada (NSERC).

Extended author information available on the last page of the article

1 Introduction

Nowadays, Machine Learning (ML) integrates a growing number of industries, from transportation to healthcare and education. Recent applications of ML achieved performances similar to humans' performance on complex tasks, such as taking the bar exam (OpenAI 2023) or driving a car (Badue et al. 2021; Gitnux 2023). ML models are implemented as software components, and then integrated into other components in Machine Learning Software Systems (MLSSs). Such systems employ trained ML models to intelligently make decisions or generate output based on learned data-derived knowledge, and similar to any software systems, they need quality assurance (Gezici and Tarhan 2022).

Behind much of the recent success of ML applications are large amounts of training data and powerful computing infrastructure (Roh et al. 2019; Whang et al. 2021). While a major part of the research in ML is spent on developing better modeling techniques (Ng 2021), data preparation often is the most arduous and time-consuming task for practitioners. Indeed, researchers have reported that data scientists sometimes spend over 80% of their time preparing data (Whang et al. 2021; Neutatz et al. 2021; Deng et al. 2017; Agrawal et al. 2019). As a consequence, there is an emerging trend, referred to as Data-Centric AI (DCAI) to steer research to focus on improving datasets instead of models for ML problems. In the last few years, various initiatives aimed at stimulating involvement in the DCAI trend have been proposed. For example, in August 2021 the first DCAI competition (Ng et al. 2021) was organized. During this competition, participants were challenged to improve a model's performance by only modifying the dataset it was trained on. Following this competition, a group of researchers released the DataPerf benchmark (Mazumder et al. 2022), a suite of data-centric benchmarks for different data tasks such as data cleaning and data debugging.

Amongst the different steps forming data preparation is a central task that consists of detecting and removing data errors, namely data cleaning (Zha et al. 2023). Previous studies have often overlooked it assuming that the datasets used in their experiments are devoid of errors. However, as reported by a previous study (Northcutt et al. 2019), large public datasets considered for a long time to be error-free such as ImageNet (Russakovsky et al. 2015) contain erroneous labels. As a consequence, there is an increasing interest in developing data-cleaning approaches to improve machine learning performance. Data cleaning is a problem tackled by the database community for a long time (Rahm et al. 2000; Fox et al. 1999; Mayfield et al. 2010) and, recently, researchers started studying how ML can be leveraged to clean data.

In this study, we summarize the latest approaches in Data Cleaning for ML (DC4ML) and ML for Data Cleaning (ML4DC). For simplicity, we refer to the union of DC4ML and ML4DC as data cleaning and ML (DC & ML). To reach that goal, we adopt the Systematic Literature Review (SLR) approach, which differs from traditional literature reviews by its strict and rigorous methodology. The SLR methodology ensures that the literature review is exhaustive and devoid of biases when collecting and summarizing information (Kitchenham 2004). With

the increasing amount of work published on DC & ML (see Sect. 5), we believe that there is a need to summarize the latest approaches in the field. A replication package of our study is available on our public GitHub repository (Côté et al. 2023). In total, we summarize the content of 101 papers and provide 24 future work directions. Our review can serve as a basis for researchers and practitioners to understand state-of-the-art DC & ML approaches and to contribute to the field.

The rest of the paper is organized as follows. In Sect. 3, the methodology followed is described. Then, in Sect. 4, the taxonomy selected to structure our review of the literature is presented. Next, statistics about the papers included in our study are detailed in Sect. 5. The review of the literature is presented in Sect. 6 and future research directions are proposed in Sect. 8. Finally, we reflect on our work and discuss threats to validity in Sect. 10. The conclusion of the paper is presented in Sect. 11.

2 Background

Recently, Machine Learning Software Systems (MLSSs) have seamlessly integrated into our everyday lives, manifesting in various forms such as recommendation systems, speech recognition, and face detection. Typically, an MLSS ingests data and utilizes ML models to autonomously make intelligent decisions based on learned patterns, associations, and data-derived knowledge (Martínez-Fernández et al. 2022). These ML models function as software components, interacting with other software components within MLSSs. Similar to conventional software systems, MLSSs necessitate quality assurance, particularly given their increasing significance in today's society. The ramifications of erroneous or subpar decisions by such systems can range from system malfunctions to substantial financial losses or even endangerment to human life (Foidl and Felderer 2019). Assessing the quality of MLSSs presents a formidable challenge and is currently a focal point of research (Gezici and Tarhan 2022; Martínez-Fernández et al. 2022). Recent research on MLSS quality underscores the importance of encompassing various dimensions of quality (Martínez-Fernández et al. 2022; Studer et al. 2021), not solely focusing on prediction accuracy.

The quality of an MLSS is contingent upon numerous factors. In MLSSs, both the code (representing the implemented model) and the training data play pivotal roles in shaping system behavior (Felderer et al. 2019; Whang et al. 2023). Compared to traditional Software Engineering (SE), the logic employed by ML models is learned from the data instead of being coded, so data quality is key to achieving good model quality. Consequently, ensuring high data quality and validating the integrity of data becomes imperative requirements in such data-dependent systems as the quality of the trained model impacts the overall quality of the system (Braiek and Khomh 2020). Data cleaning is the task of detecting or repairing corrupted, duplicate, incomplete, inaccurate, or noisy records from the dataset, to improve the quality of the dataset. This improves the quality of any potential MLSS that is trained on/uses the cleaned dataset.

Because of the importance of data quality for MLSS, there is an increasing interest in developing data-cleaning approaches to improve ML performance. Tools to address label errors in datasets, such as the platform proposed by *snorkel.ai* (Team 2024), are gaining traction in the industry. Large companies are now proposing their own tools to improve dataset quality, such as IBM's tool "Data Quality for AI" (Patel et al. 2022). In parallel, in the research community, there is a growing number of studies trying to summarize the latest advancement in this fast-moving field (Neutatz et al. 2021; Ilyas and Rekatsinas 2022; Thirumuruganathan et al. 2020; Roh et al. 2019; Chai et al. 2022).

While data cleaning plays an increasingly important role in ML, the latter can also be used to enhance data cleaning processes. Data cleaning is a problem initially tackled by the database community for a long time (Rahm et al. 2000; Fox et al. 1999; Mayfield et al. 2010) and, recently, researchers started studying how ML can be leveraged to clean data. Integrating ML into data cleaning systems presents an attractive opportunity for researchers and practitioners in the field since it has the capacity to simplify current cleaning processes. Instead of manually devising a set of cleaning rules and being burdened with its maintenance, ML offers the possibility of automatically learning these rules from a dataset. Additionally, previous data cleaning efforts can be easily integrated into any ML model as model features, as mentioned in Ilyas and Rekatsinas (2022). Finally, the use of ML for data cleaning enables data cleaning systems to benefit from the advancements in the field of ML and opens up many new research opportunities. Thus, as much as ML can benefit from data cleaning, data cleaning may also profit from ML.

3 Methodology

In this section, we describe the methodology we followed to conduct our study. We provide an outline of it in Fig. 1. Our methodology is divided into three phases: (1) paper collection, (2) paper selection, and (3) results analysis. We collected papers on 17 October 2022. For each step in the figure, we indicate (1) the section where this step is explained (in the lower left corner of the box) and (2) the number of papers left after the step (in the lower right corner of the box). To conduct our study, we followed the guidelines defined by Kitchenham (2004) for systematic literature reviews. The author described best practices for conducting systematic reviews in SE and is based on existing guidelines from medical research, where empirical research methods are mature and rigorous. As recommended by the authors Kitchenham (2004), we started by defining our research question (Sect. 3.1). We then generated our search query (Sect. 3.3) and collected papers from academic databases (Sect. 3.4). We filtered out papers that were irrelevant to our study based on selected criteria and quality control questions (Sect. 3.5). Additional relevant papers were found using snowballing (Sect. 3.6). To ensure an accurate extraction of data (Sect. 3.7), we designed and used data extraction forms, following Kitchenham (2004) recommendations.

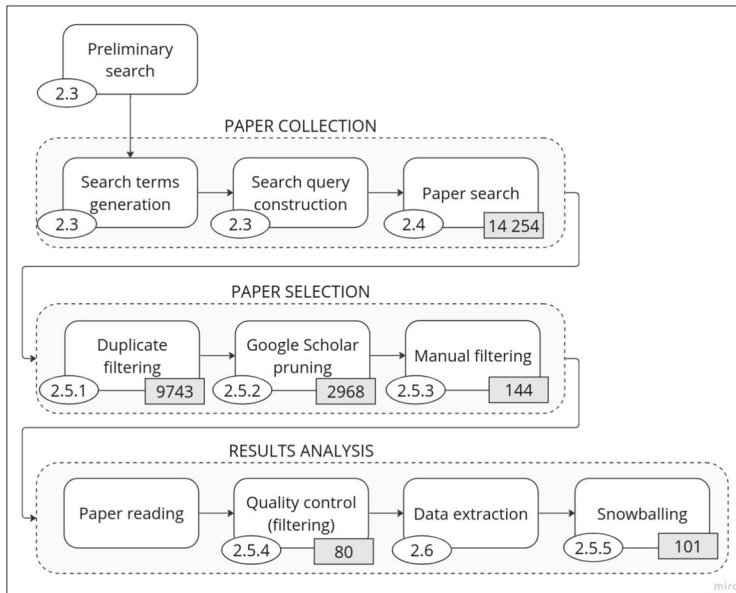


Fig. 1 Outline of our methodology

3.1 Research questions

As we mentioned earlier, the goal of this study is to conduct a systematic review of DC & ML. Thus, we define the following research questions.

RQ1: *What are the latest data cleaning techniques in DC & ML?* As mentioned before, a significant portion of practitioners' time when building machine learning systems is spent on data. In parallel, ML has made significant progress in the last ten years and is applied to a growing number of applications. Applying ML to data cleaning has the potential of reducing the amount of time spent on data by practitioners, as well as improving the accuracy of data cleaning processes. Conversely, data cleaning can play an important role in improving a model's performance. By answering this RQ, we provide researchers with an understanding of the current data cleaning approaches in DC & ML.

RQ2: *What are future research opportunities in data cleaning and ML?* In this RQ, our objective is to identify and highlight promising research directions in the field of data cleaning with machine learning.

```

("machine learning" OR "deep learning" OR "neural network" OR "neural networks" OR
 "reinforcement learning" OR supervised OR unsupervised)
AND
(
  ("data cleaning" OR "data cleansing" OR "data scrubbing")
  OR ("data repairing" OR "data repair" OR "error repair" OR "error repairing")
  OR ("confident learning" OR "label cleaning")
  OR ("error detection" AND (tab* OR cell* OR row* OR image* OR text*))
)

```

Fig. 2 The search query used to fetch papers from all academic databases used in our study except Google Scholar

3.2 Scope

For a long time, data cleaning was mainly applied to tabular data (i.e., data stored in relational databases), and surveys on data cleaning focused primarily on this type of data (Chu et al. 2016a). Nowadays, data cleaning techniques for various data types such as images (Ponzio et al. 2021), text (Klie et al. 2022), graph data (Cheng et al. 2022), and sound (Hwang and Kim 2022) are developed. The strategy used to clean data in these techniques may significantly vary from one data type to another because of the particularities of each data type. In our survey, we focused on the data types that are the most commonly processed by ML applications in practice. We used comprehensive surveys and books from the ML community to identify them (Sarker 2021; Pouyanfar et al. 2018; Goodfellow et al. 2016; Côté et al. 2023). Hence, we limited ourselves to tabular, image, and text data.

3.3 Search terms selection

As one of the first steps of the SLR, we conducted a preliminary search to help us identify the most relevant keywords to our study, as recommended by Kitchenham (2004). Hence, we read existing survey of the literature (Ilyas and Chu 2019; Neutatz et al. 2021; Ilyas and Rekatsinas 2022; Thirumuruganathan et al. 2020; Roh et al. 2019; Whang et al. 2023; Chai et al. 2022) and highly cited works (e.g., Rekatsinas et al. 2017; Krishnan et al. 2017). Then, we defined a comprehensive search query to gather the relevant literature from academic databases. Our final search query is described in Fig. 2. In the following, we describe how we selected keywords based on the main topics of our review: ‘ML’ and ‘data cleaning’. Then, we explain how we adapted our query for one of the academic databases we used, Google Scholar (GS). Contrary to other databases, GS had peculiarities that forced us to modify our search query.

- *Machine learning* ML is a field devoted to enabling machines to learn from data (Wikipedia 2023a). We leveraged previous SLRs (Tambon et al. 2022; Kang et al. 2020; Azeem et al. 2019) on different topics in ML to build an initial list of keywords for ML. Our final list of terms is: *machine learning*, *deep learning*, *neural network*, *reinforcement learning*, *supervised* and *unsupervised*.

```
"machine learning" OR "deep learning" OR "neural network" OR "reinforcement learning" OR
"supervised" OR "unsupervised"
"confident learning" OR "label cleaning" OR "error detection"

"machine learning" OR "deep learning" OR "neural network" OR "reinforcement learning" OR
"supervised" OR "unsupervised"
"data cleaning" OR "data cleansing" OR "data scrubbing" OR "data repairing" OR "data
repair" OR "error repair"
```

Fig. 3 The search query used to fetch papers from Google Scholar

- *Data cleaning* Data cleaning refers to the activity of detecting and repairing errors in data (Ilyas and Chu 2019). We used the term *data cleaning* and its synonyms *data cleansing*, *data scrubbing* to conduct our searches. Additionally, we added keywords for both processes of data cleaning: error detection and error repair. For the former one, we used the keywords: *data repairing*, *data repair*, *error repairing*, *error repair*. For the latter one, we only used one keyword: *error detection*. After running the search query with the aforementioned keywords, we initially obtained a lot of irrelevant papers in the results. The reason is that error detection is a very broad topic that is not only limited to the detection of errors in data¹. Thus, in an attempt to filter out studies that do not propose an approach to detect data errors, we limited the results to papers that are also mentioning a data type. As mentioned in Sect. 3.2, we are focusing on three common data types: tabular, text, and image data. Hence, we used the following terms to limit the papers on error detection: *tab**, *cell**, *row**, *image** and *text**. Finally, because we are interested in data cleaning for ML (and ML datasets have labels), we added the following keywords: *label cleaning* and *confident learning*.

3.3.1 Adaptations for google scholar

As opposed to other academic databases, GS has many peculiarities that require us to adapt our search pattern. Our final search pattern can be visualized in Fig. 3. Below, we describe GS's peculiarities and explain how we adapted the query.

1. All queries are truncated to 256 characters.
2. Parentheses are not a priority operator.
3. *OR* operators have a higher precedence than *AND*.
4. *AND* operators are implicitly added between terms that are not operators (e.g., terms that are not *OR* or *AND*). Hence, the search query *data cleaning* is equivalent to *data AND cleaning*.

In order to address the first constraint, we split the query in two and ran each part separately. For the same reason, we also removed any *AND* operators, since they are redundant and added automatically (constraint 4). As parenthesis do not have

¹ For example, "A modular edge-/cloud-solution for automated error detection of industrial hairpin weldings using convolutional neural networks" is a paper that was included in the results but not relevant to our study since it is not a data cleaning approach.

their usual effect (constraint 2), we also removed them from our query. This prevents any nested Boolean operator. Hence, we removed the conjunction we described in Sect. 3.3 to limit the number of results on error detection.

3.4 Paper collection

In the following, we describe how we collected papers from academic databases using the query described in Sect. 3.3. Then, we describe the separate paper collection process we followed for GS, because of the limitations of that database. Similarly to the other works (Tambon et al. 2022; Kang et al. 2020; Alsolai and Roper 2020), we conducted the search in the following academic databases:

- Google Scholar²
- Engineering Village³
- Web of Science⁴
- Science Direct⁵
- ACM Digital Library⁶
- IEEE Xplore⁷

As shown by the recent publications (Neutatz et al. 2021; Ilyas and Rekatsinas 2022), DC &ML is a relatively new topic. Hence, when running our search query, we limited the results to papers published between 1 January 2016 and 17 October 2022 (the later date being when we ran the query). On Engineering Village, we used the duplicate removal feature, by keeping the results from Compendex over Inspec.

3.4.1 Adaptations for google scholar

Unlike other databases, GS does not provide a way to export records in bulk, which discourages mining techniques. Thus, to alleviate the task of collecting paper, we used the tool “Publish or Perish”.⁸ Similarly to Tambon et al. (2022), we fetched, for each year, the 1000 most relevant records to our search query according to GS. As mentioned in Sect. 3.3.1, our initial query had to be divided into two parts because of GS’s limitations. Thus, we separately ran both queries, for every year for a period of six years, and collected 1000 papers per year, for a total of 12,000 papers (2 queries $\times$ 6 years $\times$ 1000 papers = 12,000 papers).

² <https://scholar.google.com/>.

³ <https://www.engineeringvillage.com>.

⁴ <https://webofknowledge.com>.

⁵ <https://www.sciencedirect.com>.

⁶ <https://dl.acm.org>.

⁷ <https://ieeexplore.ieee.org>.

⁸ Harzing, A.W. (2007) Publish or Perish, available from <https://harzing.com/resources/publish-or-perish>.

Table 1 Number of papers retrieved from the academic databases

Database	Number of papers
Google scholar	12,000
Engineering village	1224
Web of science	511
Science direct	135
ACM digital library	89
IEEE Xplore	295
Total	14,254

3.5 Paper selection

In this section, we explain the steps we followed to filter out irrelevant results. Table 1 shows the number of papers obtained from each academic database prior to any kind of filtering. We provide the list of papers retained after each step of the paper selection process in our replication package (Côté et al. 2023).

3.5.1 Duplicate filtering

We uploaded all the titles of papers obtained from the various databases in Zotero.⁹ We leveraged the duplicate removal functionality of the tool and found 3742 duplicates in GS' papers and 769 in the results of the other databases. Thus, after this step, 8258 papers from GS were retained, and 1485 for the other databases.

3.5.2 Google scholar pruning

While GS returned a lot of results, a considerable amount of them are not relevant to our study, for two reasons. First, a lot of irrelevant papers are included because of the search engine's limitation which we describe in Sect. 3.3. Second, GS only searches keywords in the full text of the paper or in the title. Thus, papers irrelevant to our study that mentioned data cleaning and ML in the papers' body (e.g., as key/index works) matched our search query. While we could not address the first issue in any other way than manually inspecting and then filtering papers, we devised an approach for the second issue. We managed to only keep papers whose title, abstract, and author keywords matched the search query, similar to what was done by a similar study (Tamboon et al. 2022). We did so using a Python script we implemented which is available in our replication package (Côté et al. 2023). After this step, 1862 results from GS remained. We merged this set of papers together with the 1485 papers coming from the other databases. We removed duplicated papers from the merged set, resulting in a total of 2968 papers.

⁹ <https://www.zotero.org/>.

3.5.3 Inclusion and exclusion criteria

From this step onward, papers were filtered manually. One author went through the whole set of papers and filtered out the ones irrelevant to our study. A second author verified the resulting set of papers. Any leftover duplicate was removed. For this step, the following inclusion and exclusion criteria were considered:

Inclusion criteria:

- Papers proposing an approach to detect or fix errors in an ML dataset.
- Papers presenting an approach to detect or fix errors in data while leveraging ML techniques.

Exclusion criteria:

- Papers whose data cleaning approach is specific to their problem (i.e., the technique does not generalize).
- Papers on data cleaning for other data types than tabular, image, or text data, such as sound data.
- Papers on detecting or fixing any type of attacks on an ML model through data (i.e., data poisoning and adversarial examples).
- Papers that can not be accessed free of charge through our institution's subscriptions.
- Secondary works (e.g., surveys and review papers), research proposal, workshop, letters, undergrad theses, or position papers.
- Papers that are not written in English.
- Papers on text data cleaning for other languages than English.

After this step, 124 papers remained.

3.5.4 Quality control assessment

While most papers are relevant to our study, not all of them are able to answer our RQs. Thus, based on a previous study (Tambon et al. 2022), we defined a set of questions to filter out papers with low quality. We also defined a second set of questions to filter out any remaining paper that is irrelevant to our study.

Quality control questions:

- Q1.1 Is the objective of the research clearly defined?
- Q1.2 Is the context of research clearly defined?
- Q1.3 Does the study bring value to academia or industry?
- Q1.4 Are the findings clearly stated and supported by results?
- Q1.5 Are limitations explicitly mentioned and analyzed?
- Q1.6 Is the methodology clearly defined and justified?
- Q1.7 Is the experiment clearly defined and justified?

Relevance control questions:

- Q2.1 Does the paper present an approach to detect or fix errors in an ML dataset?
Q2.2 Does the paper present an approach to detect or fix errors in data using ML techniques?

Each paper is considered by a reviewer who assigns it a score based on how well it complies with the questions. Papers under a certain threshold are considered by a second reviewer who repeats the process followed by the first reviewer. If the second reviewer's score is also under the threshold, the paper under consideration is excluded from the study. However, if the second reviewer assigns a score above the threshold, the reviewers have to discuss to decide whether the paper should be included or not in our study. If they are unable to reach a consensus, a third reviewer makes the final decision.

In order to evaluate a paper, reviewers rated on a scale from 0 to 2 how much they agreed with the quality and relevance control questions. For the quality control questions (i.e., Q1.X), a paper is excluded by a reviewer if the sum of the 7 questions is lower than 7. For the relevance control question, a paper is excluded if it has a score of zero for both questions (i.e., Q2.1 and Q2.2).

As a result, we excluded 44 papers. Our ratings are available in our replication package (Côté et al. 2023). After this step, 80 papers remained.

3.6 Snowballing

Snowballing refers to the act of discovering new papers through paper references (Wohlin 2014). In this study, to read relevant studies we miss, we include any paper cited by at least two papers from the filtered list of papers from Sect. 3.5.4. We partially automated this process with a script available in our replication package (Côté et al. 2023). In this script, we used the Python library Scholarly (Cholewiak et al. 2021) to fetch paper citations from GS. We evaluate the inclusion and exclusion criteria from Sect. 3.5.3 along with the control assessment questions from Sect. 3.5.4 before including a snowballed paper in our study. After this step, we found 21 new papers. Hence, in total, we collected 101 papers for examination in our study.

3.7 Data extraction

In order to reason efficiently over the large amount of information contained in the set of papers we read, we extracted the following information from each paper:

General information

- Title
- URL to the paper
- Authors
- Year
- Publication venue

Questions

- What does the data cleaning approach try to fix (e.g. missing values, dirty data, inconsistent data, etc.)?
- What are the datasets used in the experiment?
- What are the ML models used in the experiment?
- What are the quality measurements (i.e., how is the data cleaning approach evaluated)?
- Against what is the approach compared (e.g., against baselines, other approaches, etc.)?
- What is the performance of the approach, in terms of ML performance (e.g., accuracy, F1-score, etc.)?
- What is the performance of the approach, in terms of engineering aspects (e.g., resource consumption, time to completion, etc.)?
- What are the limitations of the approach?

Additionally, reviewers are asked to summarize the papers they have reviewed, in order to give a general picture of the study and to highlight any other interesting information. All extracted data is available in our replication package (Côté et al. 2023).

4 Selected taxonomy

As mentioned in Sect. 1, data cleaning consists of detecting and removing data errors. To categorize the collected papers, similar to existing studies (Ilyas and Chu 2019; Ilyas and Rekatsinas 2022), we first grouped data cleaning activities described in the papers based on the type of error they try to address (e.g., incorrect feature value, duplicates, etc.). The data error addressed for each data cleaning activity is highlighted in Table 2. Then, we grouped papers in each main category based on the strategy employed to clean data. For example, token-comparison approaches (Sect. 6.3.2) detect duplicate records by comparing their values, while latent space comparison approaches (Sect. 6.3.3) compare the embeddings that represent the records. Figure 4 illustrates the identified taxonomy. Bold font is used in the figure to indicate the number of papers included in a category. The number of papers in a category may not be equal to the sum of the number of papers in its subcategories because a paper may be classified to zero or many subcategories of a category it belongs to.

In the following, we introduce the different types of data cleaning activities, which correspond to the categories on the first level of the taxonomy in Fig. 4. The other categories will be introduced later in their respective subsections during the review (in Sect. 6).

- *Feature cleaning* Feature cleaning approaches try to detect or repair errors in the features of a record. It is a problem that has been addressed by the database community for a long time (Rahm et al. 2000; Fox et al. 1999; Mayfield

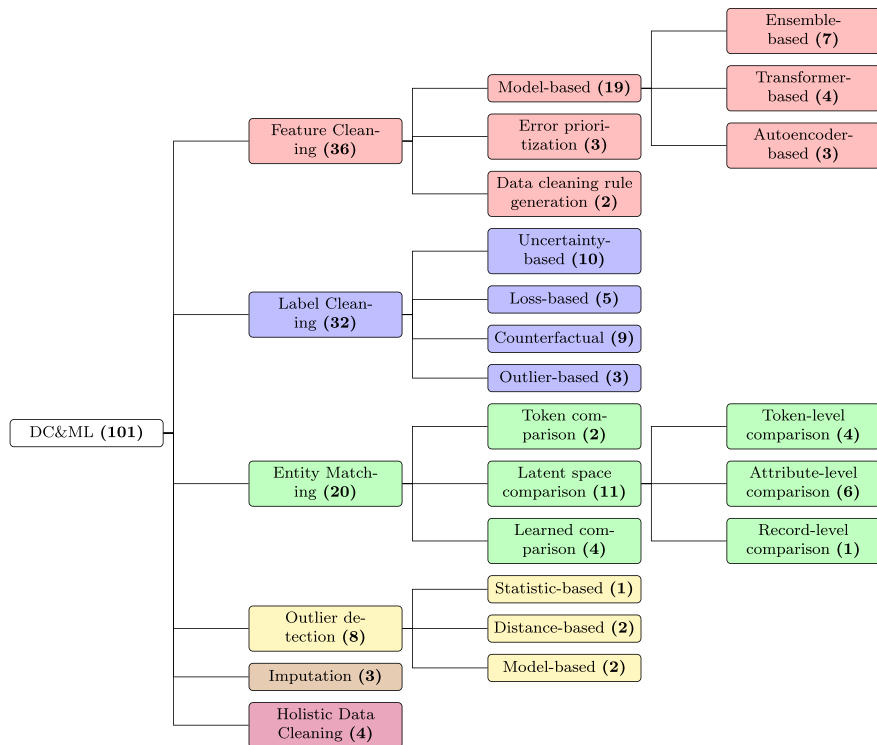


Fig. 4 Overview of the approaches discussed in this paper

Table 2 Data error type addressed per data cleaning activity

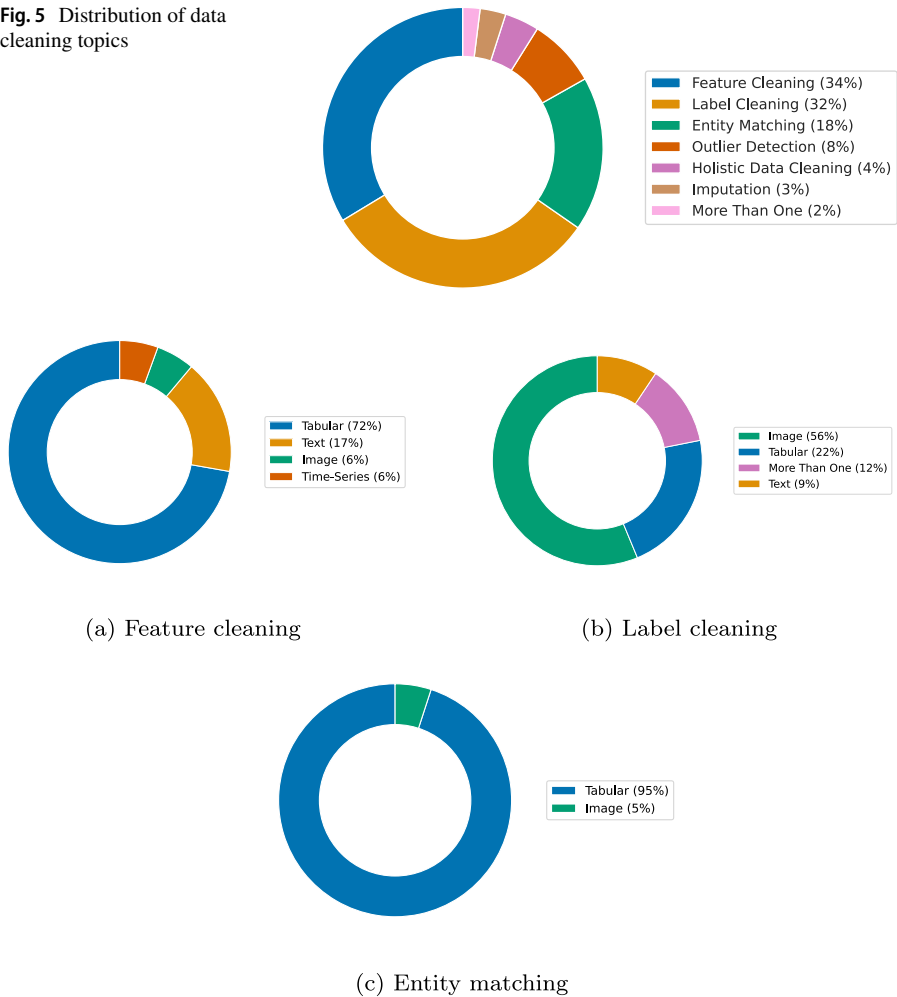
Data cleaning activity	Data error type
Feature cleaning	Incorrect feature values
Label cleaning	Incorrect label values
Entity matching	Duplicate records
Outlier detection	Out-of-distribution records
Imputation	Missing values
Holistic data cleaning	More than one error type at the same time

et al. 2010) and is generally loosely referred to as data cleaning. It should be noted that data cleaning is not limited to feature cleaning and that other activities are also considered to be data cleaning activities (e.g., entity matching or outlier detection (Ilyas and Chu 2019)). Thus, we use the term “feature cleaning” instead of “data cleaning” to refer to that data cleaning activity. Feature errors may happen for a large variety of reasons. For example, a noun may be misspelled by the person collecting data (Côté et al. 2023).

- *Label cleaning* Similar to feature cleaning, label cleaning approaches try to detect or repair label errors. Label errors may happen for a variety of reasons. For example, a record might be mislabeled because of a lack of domain knowledge (Sambasivan et al. 2021). It should be noted that label cleaning is different from label-noise robust learning (Song et al. 2023). While both approaches' main objective is to improve the performance of ML models, the strategy followed to reach that goal differs. Instead of fixing the data errors, label-noise robust learning approaches modify the learning algorithms in order to maintain the performance of ML models in the presence of label noise (Northcutt et al. 2019). Thus, label-noise robust learning is not a form of data cleaning nor is it label cleaning. We expand on label-noise robust learning in Sect. 9.3 and compare it with label cleaning.
- *Entity matching* Entity matching refers to the process of finding tuples in a database that refer to the same real-world entity (Ilyas and Chu 2019). Duplicates can be introduced in a dataset for many reasons. For example, a customer could be recorded multiple times in a database because the customer used different names at checkout (Ilyas and Chu 2019).
- *Outlier detection* An outlier can be defined as an observation that deviates so much from the other records in a dataset that it arouses suspicions that it has been generated by a different process (Hawkins 1980). Outliers may hinder an ML model's performance if they have been generated by a different process. Hence, outliers may be purged from a dataset during data cleaning.
- *Imputation* Imputation refers to assigning a value to a record's feature whose value is missing (Wikipedia 2023b). Having missing values is a frequent problem (Alimohammadi and Chen 2022) and is often experienced when data collection is manual (Lakshminarayan et al. 1999). An ML model can only ingest complete records; thus records with missing values have to be imputed otherwise they are removed from datasets.
- *Holistic data cleaning* We use the term "holistic data cleaning" to refer to the approaches that try to clean more than one type of error at a time. For example, Berti-Equille (2019) designed an RL agent that learns to clean a dataset using a set of data cleaning tools that can clean different types of errors.

In light of the identified taxonomy, we devise sub-RQs as follows to answer our RQ1 on recent data cleaning techniques in DC & ML:

- RQ1.1 What are the latest feature cleaning approaches in DC & ML?
- RQ1.2 What are recent advances in label cleaning in DC & ML?
- RQ1.3 What are the novel developments in entity matching in DC & ML?
- RQ1.4 What are the most recent innovations in outlier detection in DC & ML?
- RQ1.5 What are the latest imputation approaches in DC & ML?
- RQ1.6 What are recent advances in holistic data cleaning in DC & ML?

Fig. 5 Distribution of data cleaning topics**Fig. 6** Data types processed per data cleaning task

5 Statistical results

In this section, we provide a statistical description of the papers included in this study. Figure 5 displays the distribution of papers across data cleaning activities (which we previously defined in Sect. 4). Studies proposing an approach to clean more than one error at a time are in the category “More than one”. Generally, the papers that were included in that category proposed an approach that could be adapted for different cleaning goals (e.g., (Wang et al. 2022)). We can observe three main categories of papers, along with four minor ones. The main categories are “Feature Cleaning” (34%), “Label Cleaning” (32%), and “Entity Matching” (18%). The minor categories are “Outlier Detection” (8%), “Holistic”

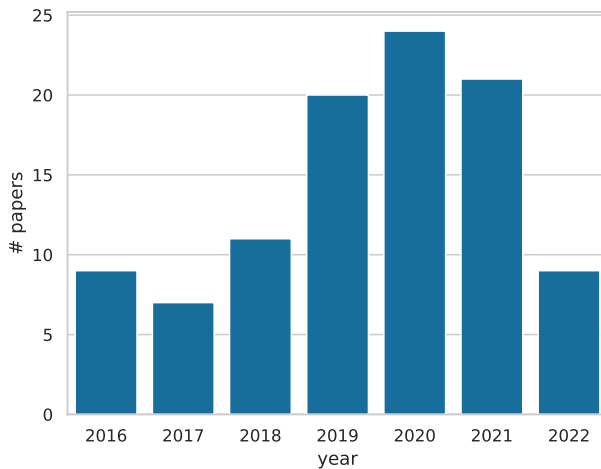
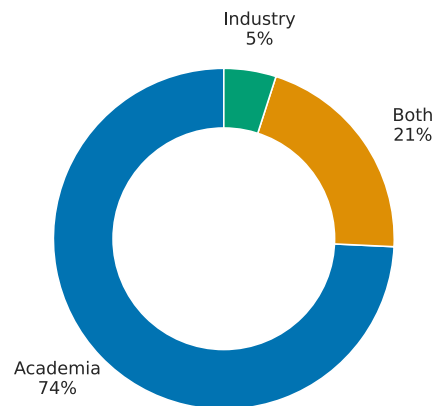


Fig. 7 Number of papers published per year

Fig. 8 Authors' Affiliation Distribution



(4%), “Imputation” (3%), and “More than one” (i.e., approaches that individually fix more than one type of error) (2%). We display in Fig. 6 the data type processed in the experiments of an approach for various data cleaning activities. We plotted this data for the top three data cleaning activities with the highest number of papers, i.e. feature cleaning, label cleaning, and entity matching. We can observe that the data type that is the most commonly processed throughout all data cleaning activities is tabular data. As can be seen in Fig. 6b, label cleaning is the activity with the highest diversity in terms of data types. In stark contrast, entity matching approaches are almost exclusively processing tabular data, as displayed in Fig. 6c. Figure 7 shows the distribution of the papers’ publication year. We can observe a generally growing trend. The number of papers in 2022 is less than in 2021 because we collected the papers during this year, in October 2022. In Fig. 8, we display the authors’ affiliation distribution. The

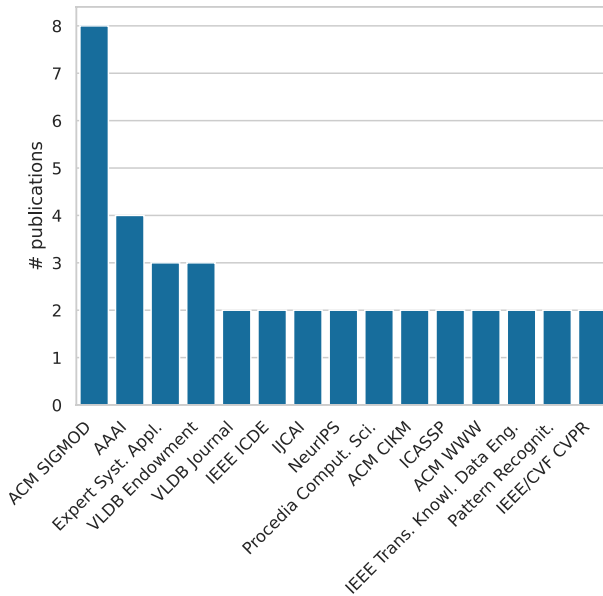


Fig. 9 Number of papers published per venue

majority of the papers included in our study have been written by researchers from academia (74%). We observe that 21% of the papers included in our study have been written by teams composed of a mix of researchers from academia and the industry. Finally, only 5% of the papers are attributed to teams from the industry. In Fig. 9, we display the number of papers published per venue. We filtered out venues that counted only one paper for readability purposes. The most recurrent venues are the ACM SIGMOD conference, the Association for the Advancement of Artificial Intelligence conference (AAAI), the Expert Systems with Applications journal, and the VLDB Endowment conference.

6 Review

In this section, we summarize current data cleaning approaches for tabular, text, and image data. We use the taxonomy described in Sect. 4 to structure our review. Hence, the following subsections discuss about feature cleaning, label cleaning, entity matching, outlier detection, imputation, and holistic data cleaning. In addition to summarizing the literature, we also provide an analysis of the metrics used for evaluation along with a comparison of the different approaches. We provide a brief description of the existing approaches along with future work recommendations at the end of every subsection.

Name	Gender	City	Country	Age	Occupation	Revenue > 70 000\$
Wintrow	Lawyer	Quebec	Canada	14	Lawyer	No
Jonathan	M	Montreal	Canada	21	Engineer	Yes
Althea	Female	Montreal	Japan	19	Doctor	Yes
Kennit	M	Ottawa	Canada	34	Professor	Yes
Jo	Male	Mtl	CAN	21	Engineer	Yes

Fig. 10 A table with feature errors (highlighted in red) (Color figure online)

6.1 Feature cleaning

In this section, we cover data cleaning approaches to detect or repair feature errors. To better illustrate the type of error cleaned by feature cleaning approaches, we provide a table with feature errors in red in Fig. 10. Because of the long-lasting efforts of the database community to clean tabular data (Rahm et al. 2000; Fox et al. 1999; Mayfield et al. 2010), the majority of the papers in this section process this data type. Thus, the following subsections present feature cleaning approaches for tabular data, except for the last one (Sect. 6.1.9). In the first one (Sect. 6.1.1), we present model-based approaches. Then, we follow with ensemble-based (Sect. 6.1.2), transformer-based (Sect. 6.1.3), and autoencoder-based approaches (Sect. 6.1.4), which all are subcategories of model-based approaches. We continue with error prioritization (Sect. 6.1.5) and data-cleaning rule generation approaches (Sect. 6.1.6). Then, we present other feature-cleaning approaches that do not belong to any of the aforementioned groups (Sect. 6.1.7). We continue with a description of approaches commonly used to improve the data cleaning approaches performance (Sect. 6.1.8) and we finish with feature cleaning approaches for other data types than tabular (Sect. 6.1.9). Finally, we describe how feature cleaning tasks are evaluated in Sect. 6.1.10 and provide a comparison of the feature cleaning approaches in Sect. 6.1.11.

6.1.1 Model-based approaches

The approaches presented in this category view data cleaning as a prediction problem. That is, a model must predict whether a value is clean or not (i.e., error detection) or what is the clean value of a dirty cell (i.e., error repair). Formally, for error detection, a model m must predict whether a record's feature value $r[A_i]$ (where r refers to a record and A_i to the i -th attribute) is dirty or not ($y \in [0, 1]$) given the full record r . Similarly, for error repair, a model m must predict the clean value y for a record's feature $r[A_i]$ given the full record r , where y 's domain is not necessarily restricted. Note that the following sections (i.e., ensemble-based, transformer-based, and autoencoder-based approaches) are also model-based approaches. In the following, when referring to records' labels, we refer to the label used by a data cleaning approach to learning to clean data.

One of the major differentiating factors between error detection approaches is their choice of feature engineering. We discerned three categories of features: (1) frequency-based features, (2) format-based features, and (3) metadata features. Similar to outlier detection approaches (see Sect. 6.4), the engineered features are generally designed to highlight the abnormal characteristics of data. A commonly used feature-engineering practice for error detection is to create features that measure how frequent a value is in a dataset (i.e., frequency-based features) (Visengeriyeva and Abedjan 2018; Neutatz et al. 2019; Heidari et al. 2019; Pham et al. 2021). For example, the authors in (Neutatz et al. 2019) measured the TF-IDF score of *n*-grams inside a cell to encode how common a cell's value is. The less common it is, the higher the chances of it being an error. Frequency-based features can be measured using the information contained in other columns (Neutatz et al. 2019; Heidari et al. 2019). For example, Neutatz et al. (2019) used the co-occurrence of values among different attributes to facilitate the error detection process. Another category of engineered features focuses on the format of the values (i.e., format-based features) (Visengeriyeva and Abedjan 2018; Neutatz et al. 2019; Heidari et al. 2019; Pham et al. 2021). Corrupted cells might not follow the syntactic format that is expected for a feature. For example, a value that represents a name should not have numbers. Pham et al. (2021) replaces all numbers and characters with symbols that are unique to them; thus focusing only on the shape of the data. For example, given the value "400\$", the encoded format could be "nnns", where "n" represents numbers and "s", symbols. Finally, the data cleaning approaches sometimes complement their feature set with metadata information (i.e., metadata features) (Visengeriyeva and Abedjan 2018; Neutatz et al. 2019). For example, Neutatz et al. (2019) indicates the data type and the string length of a cell in the feature set. For error repair, ML can be used to predict values to repair corrupted cells, similar to data imputation (see Sect. 6.5), as it was done in Ataeyan and Daneshpour (2020).

6.1.2 Ensemble-based approaches

Coined by Neutatz et al. (2021), ensemble-based approaches to data cleaning comprise every approach where a model uses the output of data cleaning tools to clean data. As it was pointed out in Abedjan et al. (2016), no single data cleaning tool is the best in every situation and no tool can detect all errors. Leveraging ensembles of data cleaning tools opens up the possibility of powerful approaches to clean errors since they could potentially detect and repair a larger diversity of errors (Rekatsinas et al. 2017; Visengeriyeva et al. 2016; Mahdavi et al. 2019). Similarly to ensembling techniques with ML models, ensemble-based approaches for feature cleaning have an ensemble of data cleaning tools, which we refer to as the "base" data cleaning tools. The outputs of the base data cleaning tools are ingested by one singular model making the final data cleaning prediction, which we refer to as the meta-model.

Usually, the base data cleaning tools are simple cleaning tools developed without using ML techniques. The most commonly used ones are integrity constraints (Rekatsinas et al. 2017; Mahdavi et al. 2019; Visengeriyeva and Abedjan 2018; Visengeriyeva et al. 2016), matching dependencies (Rekatsinas et al. 2017; Mahdavi et al. 2019), outlier detection (Rekatsinas et al. 2017; Mahdavi et al. 2019;

Visengeriyeva and Abedjan 2018), and pattern violation detection (Mahdavi et al. 2019; Visengeriyeva and Abedjan 2018). For data repair, simple algorithms that memorize transformations between dirty and clean data are sometimes used (Mahdavi and Abedjan 2020; Wang et al. 2022). These are designed specifically for ensemble-based approaches and can only suggest good repairs collectively. Hence, they are generally not viable data repair tools independently. Mahdavi et al. define three types of such tools: (1) value-based models, (2) vicinity-based models, and (3) domain-based models (Mahdavi and Abedjan 2020). Models in the first category only consider the corrupted value when suggesting repairs (Mahdavi and Abedjan 2020; Wang et al. 2022). For example, given a misspelled string such as “a dg”, the tool could suggest “a dog”. On the contrary, vicinity-based models leverage the context of the corrupted value to predict its repair. For example, given the “city” attribute of a record is set to “Tokyo”, then we know that the “country” attribute should be “Japan”. Finally, domain-based models repair a value based on the domain of the attribute. For example, a domain-based repair strategy could replace a corrupted value with the attribute’s mode.

Once the base data cleaning tools have processed an example, one is left with a set of data cleaning predictions (error detection or repair) that are ingested by the meta-model for the final data cleaning prediction. We observed different ways to include the cleaning signal of the base data cleaning tools in the prediction process of the meta-model. When the meta-model of the cleaning approach is a Probabilistic Graphical Model (PGM), the base data cleaning tools are transformed into inference rules for the meta-model (Rekatsinas et al. 2017; Visengeriyeva et al. 2016). For error detection, Mahdavi et al. (2019) trains, for each feature, a model that ingests the base data cleaning tools’ binary output (i.e., dirty or not) and predicts whether a record is dirty or not. Compared to error detection, error repair with ensemble-based approaches has the additional challenge of having base data cleaning tools that may output a wide variety of values. Thus, error repair tools adopt a different approach to combine the base data cleaning tool predictions. Instead of generating a repair, the meta-model selects the repair that is the most likely to be correct in a pool of repairs suggested by the base data cleaning tools (Mahdavi and Abedjan 2020; Wang et al. 2022). In Mahdavi and Abedjan (2020), each base error repair tool suggests values to repair corrupted cells. Then, for each repair, a vector holding the confidence that the repair is correct according to each base error repairer is generated. This vector is then fed to a Neural Network (NN) that will predict how likely a repair is given the confidence of the base error repair tools. The repair with the highest score becomes the repaired version of a dirty record. Instead of evaluating repairs using the confidence score of the base tools, Wang et al. let the meta-model select the most probable repair by comparing the repaired versions of a record with its dirty version (Wang et al. 2022). To facilitate the comparison process, the records are transformed into embeddings. Then, an NN ingests the difference between the embeddings, along with the embedding resulting from the concatenation of both records. This strategy of encoding records captures both the differences and similarities between the compared records. For every considered pair, the NN will output a score, stating how likely is the proposed repair. The most likely repair

for a record is then selected. This approach was originally designed for entity matching but showed to be apt for error repair as well.

While ensemble-based approaches may seem appealing, they have a few challenges of their own. To begin with, the predictions of a meta-model can be skewed if the predictions of the base tools are correlated (Visengeriyeva and Abedjan 2018). This is simply due to the fact that a larger feature space is allocated to what is essentially the same information. The authors in (Visengeriyeva and Abedjan 2018) address this challenge by pruning out correlated tools and keeping the most effective ones. To achieve that, k-means clustering is run on the predictions of the base tools. Then, using a validation dataset, the best tool for each cluster is selected. In addition to the aforementioned issue, a large pool of base cleaning tools results in two additional problems. First, it may lead to a longer run time for the whole data cleaning process. Thus, Mahdavi and Abedjan (2021) prunes out tools that proved to be ineffective on similar datasets. The dataset similarity is measured by comparing the semantic and syntactic similarity between columns. Second, significant human labor can ensue if a large number of tools require manual parametrization. To address this problem, Mahdavi and Abedjan (2021) generates, for every tool, all configurations in a predefined set.

6.1.3 Transformer-based approaches

In this section, we present approaches that clean the features of a record using transformer models Vaswani et al. (2017a). Because of their impressive performances in the last years, there has been an interest in the research community to apply this kind of model to a wide variety of tasks (Chen et al. 2021; Dosovitskiy et al. 2020; Chasmai 2021). Two issues must be circumvented in order for these models to be used for feature cleaning. The first hurdle is in the very nature of the task: feature cleaning is different from text generation. Effectively, transformer models are designed to generate text, while feature cleaning is usually conceptualized as a classification problem. Three approaches have been suggested to address this problem. The first solution transforms feature cleaning into a text generation problem. More precisely, Narayan et al. (2022) prompts the foundation models with data-cleaning questions. For example, for error detection, the user may ask the model if there is an error in a given sample, to which the model will answer “yes” or “no”. The authors further show that very few training samples are required for the model to detect errors, from a few (few-shot learning) to none (zero-shot learning). For error repair, Tang et al. (2020) uses a transformer to predict the value of every cell of a record and replace it if it differs from the shown value. The transformer model is trained using the masked data model objective. That is, the model ingests a record with a masked value and outputs a record with the mask replaced with a value. If the value generated by the model differs from the actual value that was masked, then the example is considered to be corrupted. Instead of transforming data cleaning into a text generation task, Nashaat et al. (2021) adapts a transformer model to a classification task. A linear layer with softmax activation is added to the output of the transformer and the model is trained to minimize cross-entropy loss. Before being fine-tuned for this binary classification task, the transformer is trained using the masked data model

objective, so it learns the structure of data. The third way to use transformers for data cleaning is by using them to generate embeddings that can be used by another model for data cleaning. Wang et al. (2022) pre-trained a transformer using SimCLR contrastive learning framework (Chen et al. 2020), so similar records have similar embeddings. Then, an NN ingests these embeddings and predicts how likely a repair is given a dirty sample. The most likely repair is selected.

The second issue that must be addressed before using transformers for data cleaning is the format of the data itself. While the data cleaning approaches in this section are applied to tabular data, transformers are trained with natural language. A naive approach would feed the record as it is to the model (i.e., transform the record into a sentence by concatenating its features using spaces). However, precious information about the features' semantics would be lost, since the model does not know what a value represents (e.g., a number does not make sense if we do not know what it counts). Hence, to address that problem, the tabular records must first be transformed into sentences, and then the model must learn to understand the structure of these sentences. To inject the tabular structure of data into strings, Narayan et al. (2022) prepends each attribute value with its name before concatenating it into a string. For a record that has a feature named "City" and a value of "Montreal", then the attribute would be represented by "City: Montreal". The latter would be concatenated into a sentence with the substrings generated for the other features. To inject the semantic difference between attributes' names and their values, Tang et al. (2020), Wang et al. (2022) further prepend attributes' names with the special token "[A]" and the values with "[V]". Extending on the previous example, we would have "[A]City [V]Montreal". Once tabular data is converted into strings, the model must learn to understand the format of the data. Similar to what is usually done with large language models (LLM), the masked data model objective is used in Nashaat et al. (2021), Tang et al. (2020) to train transformers in an unsupervised way. The model is trained to predict the missing value in a record's entry, which enables it to unlearn its assumption of processing natural language data and learn the structure of the dataset at hand. Wang et al. (2022) leveraged contrastive learning to train the encoder of a transformer to generate embeddings that are close in the latent space for similar records, and far for different records.

6.1.4 Autoencoder-based approaches

The data cleaning approaches in this section use autoencoders to detect or repair records. Autoencoders are a type of ML model trained to output the data they receive as input while being constrained to represent the data in a latent space with fewer dimensions than the input (Bank et al. 2020).

Mauritz et al. (2021) argues that autoencoders have denoising capabilities because of the compression process. Representing the input data into a lower dimension forces the autoencoder's intermediate representations to contain only the most important information necessary to recreate the input data. As a result, noise is removed from the data. Hence, they use the innate noise removal capability of autoencoders to repair records. To do so, they first transform records into probabilistic ones (i.e., records whose features have a probability distribution over potential

values). Then, they feed the record to the autoencoder, which outputs another probabilistic record. The result can be transformed back to non-probabilistic data by picking the most likely value for each feature. Tonolini et al. (2020) proposes a variant of variational autoencoders (VAE) architecture for noise removal that does not suffer from the posterior collapse problem (when the model confidently suggests a small number of reconstructions). Authors employ a three-step process to clean samples using their autoencoder. First, an example to clean is ingested by the model and a latent distribution is generated by the model. Second, multiple latent vectors are sampled from that latent distribution. Third, examples are generated from each latent vector and averaged to generate the final clean value. Instead of using the autoencoder to generate values to repair errors, Liu et al. use the reconstruction loss of an autoencoder to detect corrupted records Liu et al. (2022). Records that have a high loss during the first epochs of training are flagged as erroneous. The study further shows that poisoned samples generally have the lowest loss; hence their method can also be used to detect such records.

6.1.5 Error prioritization approaches

Instead of directly cleaning data, the approaches covered in this section try to identify and prioritize the records that should be cleaned by a human expert.

Authors in Zhang et al. (2018b) leverage the knowledge stored in Wikipedia to clean data. More precisely, by generating questions for a QA engine that understands natural language and whose answers are Wikipedia pages, they are able to verify the values in a dataset by comparing them with what is on Wikipedia. If a value conflicts with what the engine has returned, the records are sent to a human for review. Instead of identifying records that are potentially erroneous, authors in Karlaš et al. (2020) search for records that would have the largest impact on a model's performance if cleaned. They argue that data cleaning efforts are wastefully spent on samples that do not alter a model's performance. Thus, the authors propose an approach that tries to find samples that, once cleaned, will render the next iterations of cleaning less useful (since the next iterations will not alter a model's performance). Similarly, Krishnan et al. (Krishnan et al. 2016) propose an approach to prioritize the samples sent to a human for review. A model is first trained on a dataset (for the prediction task of the dataset, not for data cleaning). Then, samples that would trigger the largest weight update if the model was retrained on them after being cleaned are prioritized. The weight updates are estimated using previous examples with similar feature values.

6.1.6 Data cleaning rule generation approaches

The approaches included in this category use ML to infer data cleaning rules that can be used to clean a dataset. Data cleaning rules are typically expressed using integrity constraints, a technique from database schema design (Abiteboul et al. 1995). Integrity constraints can express a wide variety of logical clauses, such as: "two persons living in the same city must live in the same country" or "two persons can not have the same phone number" (Ilyas and Chu 2019).

Ge et al. (2020) proposes a hybrid data cleaning framework, named MLNClean, that leverages Markov Logic Networks (MLNs) to supplement integrity constraints and makes it possible to clean unknown data errors. MLNClean first uses MLN to generate a set of probable data cleaning rules with their corresponding probabilities. These rules are then leveraged to generate repair candidates. The repair candidate that is the most likely (based on the likeliness of the data cleaning rules that generated it) and that differs the least from the original value is then used to repair an observed value. In another work (Guo and Rekatsinas 2019), authors propose AutoFD, a framework for Functional Dependency (FD) (a type of integrity constraint) discovery based on structure learning techniques in PGMs (Guo and Rekatsinas 2019). A PGM is introduced to capture dependencies that FDs introduce among attributes in a dataset, revealing that learning the graph structure of this model is equivalent to discovering FDs.

6.1.7 Other approaches

In this section, we cover other feature cleaning approaches for tabular data that are not included in the previous categories.

- Krishnan et al. (2017): Similar to ensemble-based feature cleaning, their work uses an ensemble of feature cleaning tools. However, contrary to the aforementioned category, their goal is not to clean data but rather to directly generate a model that would have been trained on clean data. To do so, all feature cleaning tools are individually applied to a dataset, generating different versions of it. Then an ML algorithm trains a model for each processed dataset. The models' predictions are combined together using boosting (Wikipedia 2023d). As a result, the predictions of the ensemble are influenced by all the data cleaning tools.
- Lew et al. (2021): Authors propose a new domain-specific Probabilistic Programming Language (PPL), namely PClean that is tailored for data cleaning. Similar to Rekatsinas et al. (2017), PClean leverages dataset-specific knowledge. Authors introduce a domain-general nonparametric prior on the number of latent objects and their link structure in the dataset. Code written by PClean customizes this prior probability via a relational schema (for relational dataset) and via generative models for objects' attributes. Inference in PClean is based on a novel Sequential Monte Carlo (SMC) algorithm, to initialize a latent object database with plausible guesses, and novel restoring updates to fix mistakes.
- Oliveira et al. (2020): Their paper proposes an improvement over HoloClean (Rekatsinas et al. 2017). Most of the data cleaning approaches consider that data cleaning is only applied once, before training a model. This does not reflect how data is cleaned with real ML software systems since new training data is continuously generated. Hence, they propose a few enhancements to HoloClean (Rekatsinas et al. 2017) to make it more efficient in that context. Notably, they avoid retraining the data cleaning model if the distribution of incoming data does not change.

6.1.8 Common techniques to improve performance

In this section, we cover various techniques commonly used to improve a model's performance for model-based approaches, namely, data augmentation, semi-supervised techniques, and active learning.

6.1.8.1 Data augmentation Data augmentation is a technique to generate synthetic new examples by applying transformations to existing data (Hernández-García and König 2018). Data augmentation is used in feature cleaning approaches to address the class imbalance problem; training datasets for feature cleaning have a larger amount of labeled clean records than labeled dirty records (Nashaat et al. 2021; Heidari et al. 2019). In other words, in labeled datasets, there is generally more clean data than dirty data and dirty data is needed for the ML approach to learn to detect dirty data (error detection) or to learn to clean it (error repair). Hence, data augmentation tries to address that problem by generating more labeled dirty data. To do so, data augmentation techniques transform clean data into dirty data by applying plausible corruption mechanisms (i.e., the error induced into the record is plausible given the dataset). In order to define a data augmentation strategy, two components must be defined: the process that transforms clean data into dirty data and the process that selects data transformations (i.e., assigning a probability to each transformation). Instead of manually crafting data transformation rules, Nashaat et al. (2021), Pham et al. (2021), Heidari et al. (2019) learn them from the data. In Nashaat et al. (2021), Gestalt Pattern Matching (Tawfik and Spruit 2020) is used to find the non-matching substrings between the clean and dirty versions of a text feature. These substrings are then used to form a new mapping, from the clean substring to the dirty one. Similarly, Pham et al. (2021), Heidari et al. (2019) adopt a hierarchical pattern matching approach to learn valid transformations. Once data transformation functions are defined, the data augmentation approach must build the function that selects the data transformation that will be applied to a record. In other research works Nashaat et al. (2021), Pham et al. (2021), Heidari et al. (2019), authors sample a valid transformation following the empirical distribution of transformations (i.e., how often each transformation was used empirically).

6.1.8.2 Semi-Supervised Techniques Semi-supervised learning is a branch of ML that uses unlabeled data (in addition to labeled data) to build better models (Pise and Kulkarni 2008). The semi-supervised techniques presented in this section use the labeled data to label the unlabeled data. In order to generate training data for its data augmentation approach, Heidari et al. (2019) fits a Naive-Bayes model that, similar to imputation, predicts a clean value for each feature of a record. The most confident predictions (i.e., with over 90% confidence) become the clean versions of the observed values. Pham et al. (2021), Mahdavi et al. (2019) use label propagation to increase a labeled dataset size for error detection. Label propagation consists of assigning the label of a record to a similar unlabeled record Zhu and Ghahramani (2002). The challenge with this approach is to devise an accurate measure of similarity between records to avoid mislabels. Pham et al. (2021) considered two feature

vectors to be similar if all their attributes did not differ by more than a threshold. Similarly, Mahdavi et al. (2019) used hierarchical clustering (Aggarwal Charu and Reddy Chandan 2013) to find groups of similar feature vectors. Records in the same cluster share the same label.

6.1.8.3 Active Learning Active learning refers to a situation where a ML algorithm can iteratively query an oracle (e.g., a human) to label new data points (Wikipedia 2023c). In the following, we show how some studies used active learning to achieve superior performance. Neutatz et al. (2019) relies on the confidence score of the data cleaning models to select the cells to send to a reviewer. Because they ask reviewers to clean cells, not tuples, their acquisition function (i.e., the function that selects records) starts by selecting a feature to clean (i.e., a column of a dataset), then picks a batch of cells inside that column to clean. The column selected in the first step is the one where the model is, on average, less confident of its predictions. The cells are then selected using the query-by-committee algorithm (Freund et al. 1997). Pham et al. (2021) uses two different models for data cleaning and active learning. The model used for active learning is implemented using Probabilistic Soft Logic (Bach et al. 2017) and, similar to the data cleaning model, tries to predict whether cells are dirty or clean. Records that are more likely to be dirty are sent for review. Finally, for error repair, Mahdavi and Abedjan (2020) prioritizes tuples with a lot of errors that are common in the dataset. The goal is to create a training dataset that covers most types of errors in a dataset. Note that the approach assumes that an error-detection tool already marked the dirty cells.

6.1.9 Other data types

In this section, we cover feature cleaning approaches for other data types than tabular. We chose to present data cleaning approaches for each data type in a dedicated part because each data type has a peculiarity that makes the data cleaning task significantly different. Thus, in the following, we describe feature cleaning approaches for time series, text, and image data, respectively.

6.1.9.1 Time Series Time series are a collection of observations made in chronological order (Wang and Wang 2019). The feature cleaning approaches in this category have to consider the ordinal nature of time series data in order to clean it.

Zhang et al. (2020a) combined auto-regressive models with the minimum repairing principle of data cleaning (Ilyas and Chu 2019) to iteratively clean data. Auto-regressive models are used to suggest values to repair anomalous observations. In their approach, an auto-regressive model suggests repairs for every observation. Only one repair is applied: the one that differs the less from the original observation. The auto-regressive model is retrained on the cleaned time series, and the procedure is repeated until the difference between the original and clean value is below a threshold or the maximum number of iterations is reached. Similarly, Akouemo and Povinelli (2017) presents an iterative approach for the detection and repair of anomalies in time series data. First, Auto-regressive with

Exogenous inputs (ARX) and NN models are trained on a time series and used to predict repair values for every value of a time series. Then, hypothesis testing over the difference between observed values and predicted values is used to detect outliers. The most abnormal observed value is replaced by the predicted value, and the process is repeated.

6.1.9.2 Text In this section, we cover techniques for detecting errors in text data. The addressed errors include grammatical, syntactical, and semantic errors.

NNs models are used for detecting and correcting grammatical errors in text. For example, in He et al. (2021b), an RNN-based model is used for detecting grammatical errors in English verbs. The model consists of two sub-RNNs such that one processes the sentence from the beginning to the target verb and the other processes the sentence from the end to the target verb. An additional layer is used to process the output of the RNNs and predict the verb which is then compared with the original verb to detect any potential error. Knill et al. approach error detection as a sequence labeling task where a Bi-directional LSTM is trained to classify each token in the input sentence as correct or incorrect within the sequence Knill et al. (2019). Wang and Tan (2020) utilize word contextual embeddings generated by BERT and concatenate them to character embeddings generated by an LSTM. They then use the BERT model, in particular, the first six layers of BERT to classify each word as correct or incorrect. Rei and Yannakoudakis (2016) investigates different NN architectures (CNN, Bi-RNN, Bi-LSTM, and their multi-layer variants). Provided with a sequence of tokens, the model calculates the probability for each token to be correct or incorrect. Among the compared models, Bi-LSTM outperforms all other models including the multi-layer Bi-LSTM variant.

To detect syntactic errors, Santos et al. (2017) proposes to train two LSTM models, one is forward and the other is backward, to predict tokens. The disagreement between the two models is used as an indication of a possible syntax error. They also attempted to propose a correction by identifying an alternative token sequence derived from the models.

The model introduced by Spithourakis et al. (2016) detects semantic errors in terms of inconsistencies between a numerical value (e.g., clinical test result) and its interpretation. When a document is received, sets of words with similar meanings are employed to generate all possible word substitutions within the document, creating a set of candidate repaired documents. Each of the candidate documents is fed to an LM to predict which one is the most likely to be correct. Likelihood ratios between candidate and original documents are then computed. Candidates with higher scores than the original indicate errors; the highest score is accepted as the correction.

6.1.9.3 Image A common distortion that can be encountered in images, particularly in those captured by autonomous vehicle perception systems is a radial blur. This type of blur arises due to the motion of the imaging system during image acquisition. Hurakadli et al. (2019) proposes a deep-learning-based pipeline to

estimate and correct the radial blur in order to improve the detection of traffic signboards in images. The proposed pipeline consists of estimation and enhancement modules. The authors designed a convolutional neural network (CNN) named CuratorNet, to estimate the point spread function (PSF) of an image with a precision of up to a second decimal point. PSF is a mathematical description of how light gets spread out or blurred across the image due to various factors in the imaging system. By applying the inverse operation of the PSF to the observed image, it can be possible to recover the original scene. To correct the image, the authors propose a convolutional autoencoder-based enhancement module that eliminates the radial blur, based on the obtained PSF estimation.

6.1.10 Evaluation metrics

Error detection and repair approaches are often evaluated like classification techniques. Hence, evaluation metrics used in the field of ML, such as accuracy, F1 score, precision, and recall are most commonly used (Wang et al. 2022; Nashaat et al. 2021; Pham et al. 2021; Neutatz et al. 2019; Heidari et al. 2019). For image and time series data, different metrics are utilized. Peak-signal-to-noise is used in image cleaning research to measure the amount of noise removed from a picture (Tonolini et al. 2020; Hurakadli et al. 2019). Regression metrics, such as mean square error, are used in time-series to evaluate how close a cleaned sample is to its correct value (Zhang et al. 2020a). Since most approaches rely on a set of labeled samples to learn to detect or repair data, a few compare the performance of their approach with varying amounts of labeled data (Neutatz et al. 2019; Karlaš et al. 2020; Nashaat et al. 2021). For example, the authors in Neutatz et al. (2019) evaluate the performance of their approach trained on datasets ranging between 20 samples to 300. By following this strategy, they can measure how effectively their approach learns to clean data. Instead of evaluating the correctness of the predictions of the data cleaning approach, a subset of studies measured the impact of data cleaning on a downstream ML model (Krishnan et al. 2016; Liu et al. 2022; Krishnan et al. 2017). Other than correctness, the execution time of the approach is sometimes measured (Mahdavi et al. 2019; Liu et al. 2022; Neutatz et al. 2019; Rekatsinas et al. 2017), since it is sometimes significant. For example, the approach proposed in Rekatsinas et al. (2017) takes over 6 h to process a 2 M-examples dataset.

6.1.11 Comparison of the approaches

Among the various types of feature cleaning approaches presented, one category stands out with its performance: transformer-based approaches. They tend to outperform approaches from other categories on a large number of datasets, for error detection and repair (Wang et al. 2022; Narayan et al. 2022; Nashaat et al. 2021). For error detection, the authors in Narayan et al. (2022) showed a prompting approach that required only 10 samples to achieve better results than state-of-the-art approaches. Previous model-based approaches achieved comparable scores with only 10 to 20 samples (Mahdavi et al. 2019; Mahdavi and Abedjan 2020). However, they leveraged data augmentation to make the most out of this limited set of training

samples. Previous autoencoder-based approaches detected errors in a dataset without any training samples, albeit with worse performances (Liu et al. 2022).

In addition to being performant and requiring a few training samples, transformer approaches require very minimal manual labor to be applied to new datasets. The same prompt formats can be used to process various datasets. In comparison, ensemble-based approaches often depend on data cleaning tools that are dataset-specific. For example, a common cleaning signal ingested by many data cleaning approaches is data cleaning rules (e.g., “a country attribute must be set to Japan if the city is Tokyo”). Thus, to achieve comparable performance to the experiments conducted in the work of (Rekatsinas et al. 2017; Heidari et al. 2019; Mahdavi et al. 2019), one must first define data cleaning rules and configure data cleaning tools before cleaning any new dataset.

While the ensemble-based approaches we reviewed in this paper are generally not as performant as transformer-based approaches, they offer the possibility of integrating the latter approaches in their ensemble of base data cleaning tools and hence do not become obsolete. However, one must be careful integrating too many data cleaning tools into an ensemble-based approach, since it might lead to longer runtimes and burden the user with the configuration and maintenance of many tools, as explained in Sect. 6.1.2.

In general, transformer-based approaches propose solutions that are more performant, consume fewer training samples, and can be easily adapted to new datasets. However, they can be surpassed at times by other approaches. For example, the authors in (Pham et al. 2021) proposed a model-based approach that does not use transformer models and which outperforms transformer-based approaches on some datasets.

Feature Cleaning: Summary and Future Directions

General existing approaches: We presented approaches to clean tabular, text, and image data. The feature cleaning approaches for text included in our study fix grammatical, syntactic, and semantic errors using variations of recurrent neural networks or transformer models (Vaswani et al. 2017b). The feature cleaning approaches for images included in our study fix image blur using Deep Learning (DL) techniques. We identified the following feature cleaning approaches for tabular data.

- *Model-based approaches:* Approaches that train an ML model to directly clean data. One may observe the following three subcategories.
 - *Ensemble-based approaches:* An ML model uses the predictions of base data cleaning tools for cleaning. The base data cleaning tools are generally simple.
 - *Transformer-based approaches:* Approaches that use a transformer to clean the features of a record. They rely on the capabilities of pre-trained LLMs.
 - *Autoencoder-based approaches:* Approaches that use autoencoders to clean data. They rely on the denoising capability of auto-encoders.
- *Error prioritization approaches:* Approaches that prioritize the records that should be reviewed by an expert.
- *Data cleaning rule generation approaches:* Approaches that generate data cleaning rules and use them to clean data.

Future directions:

- Develop feature cleaning approaches that consider the label of a record when cleaning its features. The approaches covered in our study do not leverage the special status of labels when cleaning a feature (they are not mentioned nor treated differently than other features). Potentially, labels could help clean the features of a record more efficiently than if treated as another feature. Thus, we encourage future works to explore how the records' labels can be used for feature cleaning.
 - Extend the advances done with ensemble-based approaches. Future works should experiment with a wider number of base data cleaning tools. For example, other feature cleaning approaches could themselves be included in the ensemble.
 - Explore how techniques from outlier detection can be transposed to error detection. Outlier detection and error detection are similar challenges since corrupted records often are outliers. Future studies could leverage the existing literature on outlier detection and ML (Pang et al. 2021; Nassif et al. 2021).
-

6.2 Label cleaning

In this section, we cover data cleaning approaches to detect or correct label errors. To better illustrate the type of error cleaned by label cleaning approaches, we provide a table with label errors in red in Fig. 11. For simplicity, we refer to label errors as mislabels. We do not differentiate the label cleaning approaches based on type of the data being cleaned, since the main ideas are generic. That is, any approach can be adapted to other data types if an adequate representation learning technique is used.

In the following, we describe the different categories of label cleaning techniques seen in our review. The first category uses the confidence score of an ML model to detect corrupted samples. The second one adopts a similar approach but uses the loss of a model instead of its confidence scores. The third one detects mislabels by searching for instances that degrade a model's performance.

Name	Gender	City	Country	Age	Occupation	Revenue > 70 000\$
Wintrow	M	Quebec	Canada	14	Lawyer	No
Jonathan	M	Montreal	Canada	21	Engineer	Yes
Althea	Female	Montreal	Japan	19	Doctor	Yes
Kennit	M	Ottawa	Canada	34	Professor	Yes
Jo	Male	Mtl	CAN	21	Engineer	Yes

Fig. 11 A table with label errors (highlighted in red) (Color figure online)

The fourth one uses outlier techniques to find mislabels. We also present label cleaning approaches that did not belong to any of the aforementioned categories. We describe how label cleaning tasks are evaluated in Sect. 6.2.6 and provide a comparison of the label cleaning approaches in Sect. 6.2.7.

6.2.1 Uncertainty-based approaches

A straightforward approach to label cleaning is to use the confidence score of an ML model to detect corrupted samples. After having been trained on a dataset, one can suppose that an ML model has learned the representative features of each class. Hence, if the model's predictions disagree with the assigned label for an instance (i.e., the model assigns a low probability for the labeled class), one can assume that the instance has a higher chance of being corrupted compared to other samples. However, the model may predict something different than a sample's label because of corrupted features, mislabels, or more simply because the model made a mistake. Thus, the predictions of the model can be used as a weak signal for detecting mislabels.

Once a sample is marked as being mislabeled, it must be cleaned. The cleaning strategies sometimes assume that someone will manually clean the flagged instances. When they do, either all the instances are sent for review (Rottmann and Reese 2023; Teso et al. 2021) or only the most important ones. In the latter case, one must find criteria to prioritize samples over others. Rehbein and Ruppenhofer (2017) prioritizes records for which the model's predictions have the highest entropy. Similarly, Atkinson and Metsis (2021), Atkinson and Metsis (2020) prioritize the records where the model is highly uncertain about the labeled class. In order to avoid uselessly spending humans' time, Bernhardt et al. (2022) proposed to also consider the difficulty of labeling an instance for a human. Thus, the approach considers two things when selecting the next sample to clean: (1) how much the model disagrees with the label and (2) how difficult it is to clean for a human. To measure the latter, the authors considered the entropy of a model's predictions (the higher the entropy, the more difficult labeling). Other uncertainty-based approaches (Ponzio et al. 2021; Northcutt et al. 2019; Liu et al. 2020; Ke et al. 2019) do not assume the presence of a human expert for cleaning instances; hence, they must automatically handle the cleaned instances. The straightforward strategy is to drop all the flagged instances (Ponzio et al. 2021; Liu et al. 2020). As a result, the dataset is purged from

every potentially corrupted instance. Instead of removing all potentially corrupted instances, Ke et al. (2019) iteratively removes the instances with the lowest confidence and retrains the model on the cleaned dataset until the model's confidence for all instances is above a threshold. Northcutt et al. (2019) builds a class-conditional noise matrix and suggests different methods for record pruning using that matrix. This approach uses the confidence of the model to estimate class-conditional noise. An instance might belong to another class if the model confidence for that other class is above the per-class threshold. The per-class threshold of a class j is the model's average confidence over the samples labeled j for the same class j . Having a threshold defined for every class allows the approach to be more robust to heterogeneous class probability distributions and class imbalance. When an instance is above the threshold for more than one class, the approach selects the class the model is the most certain about. The authors propose five methods to prune out records while considering class-conditional noise. One of them prunes out the records that are predicted to belong to another class (when using the per-class thresholds).

For the uncertainty-based approaches to be successful, two things must be taken into consideration. First, the model should not overfit the dataset, or else the mislabeled instances will be correctly predicted (with respect to the labeled class, not the true class) and mislabeled instances will have a high confidence score. Second, an accurate measure of the model confidence must be used. Prior work (Gal and Ghahramani 2015; Hendrycks and Gimpel 2016) has shown that the softmax score should not be used to measure the model's confidence, since it only acts as a normalization factor.

The first problem can be addressed using techniques that are robust to label noise, for example, Bernhardt et al. (2022) used Co-Teaching (Han et al. 2018) and BYOL (Grill et al. 2020). Another solution to prevent the ML model from predicting the mislabeled class of an instance is to stop the training process before the model overfits the data. Köhler et al. (2019) made the observation that the mislabeled records are learned later in the model's learning process compared to the correctly labeled ones, and, as a result, stopping the training early would enable the model to have low confidence on mislabeled instances. Leveraging that observation, Ponzio et al. (2021) proposed an approach to automatically stop the training process. After each epoch, the model's confidence for each instance's labeled class is recorded and clustered using the k-means algorithm with a k value of 2. Hence, after each epoch, a cluster with the most certain records and another with the least certain records are generated. When less than 1% of the records change the cluster from one epoch to another, the model stops training.

The second problem can be addressed using modeling techniques able to provide uncertainty scores, such as Bayesian Neural Networks (BNN) (Ponzio et al. 2021). The output of these NNs is a posterior distribution over the classes. Therefore, BNNs give a good measure of confidence when adequately trained. They are, however, complicated to implement and hard to train due to the high number of hyperparameters and computational cost (Ponzio et al. 2021). Thus, uncertainty-based approaches use different approximations of BNNs when concerned with providing a good measure of confidence. Inspired by the work of (Köhler et al. 2019), which also leverages Deep Ensemble (Lakshminarayanan et al. 2016), Ponzio et al. (2021)

approximates BNNs with Monte Carlo Dropout (MCD) (Gal 2016). MCD provides confidence measures that are numerically tractable and easier to implement than BNNs.

6.2.2 Loss-based approaches

Similar to uncertainty-based approaches, loss-based approaches posit that a model trained on a dataset will have learned the representative features of each class and, as a result, will generate predictions that are not aligned with incorrect labels. However, instead of relying on a model's confidence, loss-based approaches will detect mislabels using the loss of a model. Records for which the model has a high loss are considered more likely to be mislabeled than other records.

The authors in Huang et al. (2021), and Li et al. (2021) first train a model on a dataset, then flag any record with a high loss value as mislabeled. Flagged records are then relabeled with the model's predictions. A new model is then trained on the modified dataset, and the process is repeated using the new model and the modified dataset. The authors in Huang et al. (2019) made the observation that NNs tend to overfit on noisy examples later in the training process than clean ones. Thus, corrupted examples are identified by looking for examples with a higher loss than average during training. However, the loss values of noisy and clean samples might vary depending on how the NN is initialized, which makes the approach less reliable. To address that concern, model training is repeated several times and the loss values are averaged. The weights of the NN are periodically re-initialized by suddenly increasing the learning rate and taking large gradient steps, which makes the model unlearn. The authors in Yu and Aizawa (2020) address the challenge of detecting records not belonging to any of the classes of a dataset (noisy class). The approach adds a new class to the dataset for these samples. Then, a probability of 0.5 is assigned to the labeled class and the noisy class for every sample of the dataset. The approach iteratively trains a model and modifies the labels in such a way that the model's loss is decreased. Similarly to Huang et al. (2019), the authors use a high learning rate to avoid overfitting on noisy labels. Similarly to Picket (Liu et al. 2022) (see Sect. 6.1.4), the authors in Zhang and Tan (2019) measure the reconstruction error of autoencoders to detect corrupted samples. Examples with a high reconstruction error are flagged as mislabeled instances. Contrary to Picket (Liu et al. 2022; Zhang and Tan 2019) trains an autoencoder for each class. To repair a mislabeled instance, they measure the reconstruction error of all the autoencoders for that sample and assign the class associated with the autoencoder with the lowest error. Similarly, the authors in Salekshahrezaee et al. (2021) used and compared three ML techniques, namely principal component analysis (PCA), independent component analysis (ICA), and autoencoders for unsupervised label noise detection. With each technique, they constructed a binary classifier that identifies instances with a high reconstruction error as anomalies. Based on their results, the autoencoder-based classifier obtained the best label noise detection score.

6.2.3 Counterfactual approaches

A reason to do data cleaning is to improve a model's performance (Hara et al. 2019). This implicitly assumes that data errors reduce a model's performance. Thus, by finding dataset repairs that improve a model's performance, it is possible to find corrupted instances. In this study, we observed two types of dataset repairs: instance removal (i.e., dropping a record from a dataset) and attribute modification (i.e., modifying a record's features values, or labels). Counterfactual approaches try to find or repair corrupted records by comparing the performance of a model trained on a dataset with the performance of a model trained on the counterfactual dataset (i.e., the same dataset with some features or labels modified). In the following, we describe how the counterfactual modifications (i.e., instance removal and attribute modification) are used to detect mislabels and how they are implemented. Then, we cover the metrics used to quantify the impact of a counterfactual modification on a model's performance.

The first category of counterfactual approaches detects mislabels by comparing the performance of a model when trained on a dataset including a sample with the performance of a model when trained on the same dataset without that sample. A naive approach would train the model twice, once with the sample and another time without it. However, this can quickly become intractable as the size of the dataset increases. To address that problem, the authors in Dolatshah et al. (2018) used parallel computing. Two other techniques have been proposed to address this problem in other works. The first technique is to use ML algorithms to predict the impact of including a record in the training dataset of a model (Smyth 2020). In other words, an ML model is in charge of predicting the change in validation loss when a record is added to the training dataset of another ML model. To achieve that, Smyth (2020) records a model's validation loss at every step of the training process. Hence, each time a model learns from a sample, the model is evaluated on a validation dataset. The validation loss is averaged per sample and the average becomes each record's label. Using this new dataset, ML algorithms can be used to predict the change of validation loss for any sample added to a dataset. The number of samples used to train the model predicting deltas in validation loss is a subset of the whole dataset to clean. The second technique to approximate the impact of removing a sample from a model's training dataset is using influence functions (Suzuki et al. 2021; Wu et al. 2021; Koh and Liang 2017). Influence functions are a technique from robust statistics that can be used to estimate how much a model's parameters change in reaction to an infinitesimal change in a training point's weight in a dataset. The influence function implementation of Koh and Liang (2017) can estimate the effect of removing an instance from a dataset needing only an oracle access to gradients and Hessian-vector products of the ML algorithms. However, it is theoretically sound only for convex-loss models. To address this limitation, Hara et al. (2019) designed a novel estimator that, similar to influence functions, can estimate the change in performance if an instance is removed from the training dataset for models trained with stochastic gradient descent. In an experiment where the instances with the worst influence on a model's performance are progressively dropped, Hara et al. (2019) have shown that the approach improves the model performance more effectively

than Koh and Liang (2017). Similarly to Hara et al. (2019), (Suzuki et al. 2021) and (Wu et al. 2021) provide improvements in terms of efficiency to Koh and Liang (2017)'s implementation of influence functions, so that it can be used more efficiently for label cleaning.

The second category of counterfactual approaches detects mislabels by comparing the performance of a model when trained on a dataset including a sample with the performance of a model when trained on the same dataset with a modified version of that sample. In addition to detecting potentially corrupted records, this counterfactual approach also provides repair suggestions (the counterfactual sample). Xiang et al. (2019), Zhang et al. (2018c) formulate the search for mislabeled instances as an optimization problem where the goal is to maximize the model accuracy on an error-free dataset, by correcting the training samples' labels. Both approaches can automatically correct mislabels given an error-free test dataset.

Except for one paper in our selected papers, the counterfactual approaches included in this review evaluated modifications to a dataset using a loss function. However, as pointed out by Flokas et al. (2022), loss functions are not always a good objective function to optimize for, since influence functions can not find records corrupted by systematic noise. Indeed, removing a record corrupted with systematic noise will not significantly impact the model's performance, since other samples are affected by the same noise. Hence, records corrupted with systematic noise are not influential per se. To address this issue, Flokas et al. (2022) designed an approach that uses complaints (filed by a human) to find corrupted instances. Complaints can be filed regarding tuples (e.g., "that record should not have this label") or aggregate values (e.g., "there should be a fewer number of records that have a label"). The counterfactual approach then addresses that complaint using influence functions to find instances that, once repaired, will address the complaint.

6.2.4 Outlier-based approaches

Outliers are records that are significantly dissimilar from other records, to a point that one can believe that they have not been generated by the same process (as the rest of the data) (Wang et al. 2019). Thus, to detect mislabeled instances, outlier-based approaches for label cleaning search for records that differ significantly from the other records from a class. In Guo et al. (2018), a record is compared against every other clean record from its class using cosine distance. If the average distance is too far, then the record is marked as dirty. Because this approach does not scale to classes with many records, Lee et al. (2018) compares instances against class prototypes; a vector representing all the instances from a class. The class prototype is generated by encoding a subset of the most representative records from a class using self-attention. Self-attention allows the encoder to focus on the most important instances when generating the final embedding. Records whose cosine distance with the class prototype is above a selected threshold are flagged as dirty. The approach iteratively runs the cleaning technique and trains a model on the cleaned dataset. The first layers of the trained NN are used to generate embeddings for the data cleaning steps. Thus, every time the NN improves performance, the data cleaning

technique uses improved embeddings. Bagherzadeh and Sadoghi Yazdi (2017) proposes a method for label denoising based on Bayesian aggregation. This approach addresses the limitations of kNN-based approaches, which are not robust to high levels of label noise rate since they only have a local view of instances. A local view refers to considering the immediate neighbors or similar instances of a data point within a certain vicinity. In contrast, a global view refers to comparing a record to others that are not in its neighbor. The proposed approach combines both views to detect mislabels. The global view of data is obtained by measuring the distance of an instance to its class distribution or by evaluating the value of the probability density function of each class at an instance. The aggregation leads to a robust detection of instances with noisy labels even in the presence of high levels of label noise.

6.2.5 Other approaches

In this section, we cover other label cleaning approaches that are not included in the previous categories.

- Ekambaram et al. (2016): The authors argue that records near the decision boundary are more likely to be mislabeled by a human because they are more difficult to label. Thus, the authors propose to review support vector examples of an SVM by a human. Because there might be a lot of samples to review, they prioritize the ones that are on the wrong side of the decision boundary of another model trained on the dataset devoid of the support vector records.
- Su et al. (2021): Instead of detecting and repairing errors, they generate a synthetic dataset devoid of mislabels. To do so, the authors leverage the phenomenon of mode collapse (goo 2022) in Generative Adversarial Networks (GANs). They train a GAN to generate realistic images for a class. Because GANs are subject to mode collapse, they expect the synthetic dataset to be free of mislabeled instances, since the GAN will not have memorized the less common instances of a class (i.e., mislabels).
- Veit et al. (2017): Instead of detecting errors and repairing them in two different steps, the authors propose an approach that directly maps an example's label to the correct value. An NN is then trained to predict the correct label given an instance and its label. An identity skip-connection is between the input label and the output of the NN. Thus, its task is to map the input label to the correct value. Similar to Northcutt et al. (2019), the authors argue that label noise is often class-conditional. Thus, learning to map labels from one class to another is sensible.
- Klie et al. (2022): This study evaluates existing methods for detecting annotation errors and inconsistencies in text. Inconsistencies in this context refer to instances that should be labeled in the same manner to indicate the same type but are assigned different labels instead. The authors re-implemented 18 such methods and assessed their performance on several datasets for text

classification, token labeling, and span labeling. The experiments indicate that inconsistencies are more challenging to detect than annotation errors.

6.2.6 Evaluation metrics

The metrics used to evaluate label cleaning methods are highly similar to the ones used in feature cleaning works. Since label cleaning can be evaluated like a classification problem, metrics such as accuracy, F1 score, precision, and recall are commonly used (Rottmann and Reese 2023; Flokas et al. 2022; Smyth 2020; Zhang and Tan 2019; Rehbein and Ruppenhofer 2017). A large number of works measured the improvement of a downstream model to evaluate their approach, since improving the performance of a model is often the end goal of correcting label errors (Veit et al. 2017; Su et al. 2021; Suzuki et al. 2021; Wu et al. 2021; Dolatshah et al. 2018). In addition to measuring the correctness of the predictions of their data cleaning approaches, label cleaning works also considered other aspects, such as the execution time (Flokas et al. 2022; Zhang and Tan 2019) and memory consumption (Gemp et al. 2017) of their approach.

6.2.7 Comparison of the approaches

All types of label cleaning approaches propose viable strategies to detect mislabels. However, if one's goal is to improve ML performance, counterfactual approaches are an interesting option. Indeed, their use of influence functions enables them to directly identify the samples that are the most likely to improve the performance of an ML model. However, counterfactual approaches suffer from a few limitations. Like many other label cleaning approaches, they are not robust to systematic noise when the goal is to improve ML performance (Flokas et al. 2022). Indeed, if the goal is to detect the samples that are the most likely to improve ML performance, then the samples tainted by systematic noise will not be identified, since the systematic noise will not be removed from the dataset by cleaning only one sample. Hence, other works propose alternative ways to use influence functions. As discussed in Sect. 6.2.3, the authors in Flokas et al. (2022) let the user provide custom objectives that are less likely to be affected by systematic noise than the loss of ML models. Instead of using influence functions to detect incorrect labels directly, the authors in Teso et al. (2021) use them to find counterexamples to cleaned samples (i.e., samples that must have an incorrect label given a cleaned sample). Similar work is done in Zhang et al. (2018c). It is important to note that other label cleaning approaches are not necessarily impermeable to systematic noise. As shown in Ekambaram et al. (2016), uncertainty-based approaches (and loss-based approaches) are similarly susceptible to systematic noise in a dataset. Indeed, an ML model can learn systematic noise and have high confidence on noisy samples. In addition to having this limitation, a model may also learn any sort of noise by overfitting. We discussed mitigation techniques for this problem in Sect. 6.2.1. Hence, while all types of approaches



Name	Gender	City	Country	Age	Occupation	Revenue > 70 000\$
Wintrow	M	Quebec	Canada	14	Lawyer	No
Jonathan	M	Montreal	Canada	21	Engineer	Yes
Althea	Female	Montreal	Japan	19	Doctor	Yes
Kennit	M	Ottawa	Canada	34	Professor	Yes
Jo	Male	Mtl	CAN	21	Engineer	Yes

Fig. 12 A table with duplicates (highlighted in red) (Color figure online)

can be susceptible to systematic noise, the flexibility of counterfactual approaches allows them to define objectives less influenced by such noise.

Other limitations of counterfactual approaches stem from the selected implementation for influence functions. Previous works have highlighted the efficiency issues counterfactual approaches faced when using influence functions (Zhang et al. 2018c; Suzuki et al. 2021). To make them computationally tractable, other works have made assumptions that impose limitations on the generalizability of their approach. For example, Hara et al. (2019) proposed an approach to detect mislabels that only works for models trained with stochastic gradient descent. Similarly, the authors in Wu et al. (2021) assumed strong convexity on the models (e.g., logistic regression). Thus, because of the constraints of many counterfactual approaches, practitioners might be deterred from using them to clean the labels of datasets. On the other hand, uncertainty-based and loss-based approaches tend to be easier to implement and present fewer constraints. For example, the approach described in Northcutt et al. (2019) detects mislabel solely by inspecting a model's confidence distribution once trained on a dataset. Similarly, the authors Huang et al. (2019) propose an approach that identifies incorrectly labeled data based on the loss of the model during training. For both approaches, little to no effort is required to configure the cleaning approaches once the training procedure of an ML model is configured.

Label Cleaning: Summary and Future Directions

General existing approaches:

- *Uncertainty-based approaches*: Approaches relying on a model’s confidence to detect mislabels. Records for which a trained model predicts a different class than the one that is labeled have a higher chance of being mislabeled than other records.
- *Loss-based approaches*: Approaches relying on a model’s loss to detect mislabels. Similar to uncertainty-based approaches, records that are mislabeled are more likely to be incorrectly predicted by the model than other records (and have a high loss).
- *Counterfactual approaches*: Mislabels may hinder a model’s performance. Thus, if the performance of a model improves after a record has been removed or relabeled, then the record is most likely mislabeled.
- *Outlier-based approaches*: Records that are incorrectly labeled might differ significantly from the other records in the labeled class. These approaches rely on that observation to detect mislabeled.

Future Directions:

- Develop efficient and accurate methods to estimate the impact of removing a record from a dataset on ML performance. Current solutions may consume a lot of resources (Suzuki et al. 2021; Hara et al. 2019) or be limited to a specific type of model (Koh and Liang 2017). Improving these techniques will allow for more performant counterfactual approaches (covered in Sect. 6.2.3) while enriching the knowledge of ML explainability.
 - Combine label cleaning methods using ensembling techniques for improved performance. In Sect. 6.1.2, we described approaches that combine feature cleaning tools using ensembling techniques. Reusing the same ideas but for label cleaning could be an interesting research direction.
 - Develop accurate model confidence measure. As we covered in Sect. 6.2.1, uncertainty-based approaches rely on a model’s confidence to detect mislabels. However, using a model’s softmax score is not a good measure of model confidence as previous research pointed out (Gal and Ghahramani 2015; Hendrycks and Gimpel 2016). Future works could develop techniques to better measure a model’s confidence.
-

6.3 Entity matching

In this section, we cover approaches to detect records referring to the same real-world entity. To better illustrate the type of error cleaned by entity matching approaches, we provide a table with duplicates in red in Fig. 12. For ML, entity matching can be framed as a binary problem, where the model receives two records and must predict if they are the same entity or not. Formally, given a pair of records (t_1, t_2) , a model M predicts if the pair of records is a match or not ($y \in [0, 1]$). For conciseness, we use the word “pair” to refer to a pair of records. Furthermore, for simplicity, we call pairs referring to the same real-world identity as positive pairs, and negative pairs if they do not.

Deciding whether records refer to the same real-world entity essentially comes down to comparing records and looking for similarities and differences. Hence, when designing entity-matching approaches, researchers may bias the model into comparing records, which we refer to as the comparison bias. For example, Jin et al. (2021) compare records by looking for shared and unique words between two records. The sets of shared and unique words are used by the model to decide whether they refer to the same entity or not. Because the comparison bias plays a

central role in the design of the approaches, we grouped the approaches based on the way this bias is injected. Before the record-matching step, pairs that are most certainly not a match are filtered out of the dataset for efficiency purposes. This step is referred to as blocking.

In the following, we explain entity matching and present the approaches included in our study in four parts. We will first cover the blocking strategies. Then we present the different entity-matching approaches using ML. Afterward, we show common techniques used to improve a model's performance on entity-matching tasks (e.g., data augmentation). Finally, we describe how entity-matching tasks are evaluated in Sect. 6.3.7 and provide a comparison of the entity-matching approaches in Sect. 6.3.8. Note that all the entity-matching approaches in our review operate over tabular data.

6.3.1 Blocking

As mentioned earlier, blocking is applied before entity matching. The goal is to make the whole process of finding matching records more efficient. Instead of comparing each and every instance using the matching model, blocking places records into blocks so that entity matching is only applied to the tuples in the same blocks. This process effectively filters out pairs of records across different blocks, which have a low probability of being a match, making the whole process more efficient.

In the simplest case, hand-crafted rules can be used to filter out pairs of records that most likely are not a match. For example, for an entity matching task of companies' names, Li et al. (2020b) exclude pairs of records that do not share the same zip code or were not one of the top 20 nearest neighbors based on the TF-IDF cosine similarity on the name and address attributes. A downside of that approach is that designing blocking rules requires prior domain knowledge and can be time-consuming. Hence, Ebraheem et al. (2018) proposed an approach that is able to filter out pairs of records with minimal human involvement. To do so, Local Sensitive Hashing (LSH) (Indyk and Motwani 1998) is applied to the distributed representations (i.e., embeddings) of the tuples. Initially designed to efficiently approximate the k -nearest neighbors problem in sublinear time, LSH is used to filter out improbable matches. It hashes the distributed representations and uses the hash function to place each record into blocks, where each block holds records that are spatially close. Only pairs inside the same block can be considered for entity matching. (Ebraheem et al. 2018) generate the records' distributed representations by transforming the record's tokens into embeddings using the Glove embedding model (Pennington et al. 2014) and merging them together with a bi-LSTM (Hochreiter and Schmidhuber 1997). The pre-trained token embeddings are fine-tuned on the domain of the task using vocabulary retrofitting. Similarly, Zhang et al. (2020b) applies LSH on records' embeddings. To the difference of Ebraheem et al. (2018), they generate more than one embedding per record. The records having at least one embedding in the same block as another record will be forwarded to the entity-matching model. The other pairs will be filtered out. The embeddings are generated by fusing fastText (Bojanowski et al. 2017) embeddings using attention mechanisms. Similarly to this work, Huang et al. (2023) uses the FAISS library (Johnson et al. 2019) to efficiently

compute the k -nearest neighbors and then applies an algorithm to improve recall so that pairs of tuples that are a match are not filtered out. Essentially, the algorithm resizes the k -nearest neighbors set for each record (i.e., adjust “ k ”) so that the likelihood of the k neighbor being a match is significantly more probable than the $k+1$ neighbor.

6.3.2 Token comparison

The first category of entity-matching approaches compares records at the feature level. These approaches first extract a set of shared and unique words from the compared records, transform the words into embedding, and feed them to the model. Jin et al. (2021) creates, for each attribute, a set of shared and different words. For example, for the values “Montreal, Canada” and “Quebec, Canada”, the set of shared words would be “{Canada}” and the set of different words would be “{Quebec, Montreal}”. Similarly, Wang et al. (2020) creates a set of shared words and two other sets containing the words unique to each record. The words in each set are then transformed into embeddings using a pre-trained language model (Jin et al. 2021) or word embeddings (Wang et al. 2020). Each set of word embeddings is then condensed down to an embedding using summation (Wang et al. 2020) or attention mechanisms (Jin et al. 2021; Wang et al. 2020). As a result, an embedding is created per set of words. These embeddings are then concatenated together and fed to the last layers of the model for prediction. In addition to the aforementioned strategies to combine embeddings, Wang et al. (2020) proposes to use the embedding resulting from the set of shared words to serve as the “query” key in the attention mechanisms applied to the sets of unique words. Their intuition is that information from the shared words is useful to know the important words in the set of unique words.

6.3.3 Latent space comparison

The second type of entity-matching approach compares records in a latent space using a distance measure such as cosine distance before predicting whether a pair of records is a match or not. The records can be compared at the token level, the attribute level, or at the record level. We will refer to the embeddings used for comparison at each of these levels as token embeddings, attribute embeddings, and record embeddings respectively. Effectively, these approaches use one of the aforementioned embeddings for each record, compare the embeddings, aggregate the results of the comparison, and feed it to the entity-matching model.

6.3.3.1 Token-level comparison First, embeddings must be generated for each token. They can be generated from pre-trained embedding models (Fu et al. 2021; Mudgal et al. 2018), transformers’ encoders (Huang et al. 2023), or any other suitable technique. To generate token embedding, Li et al. (2020a) uses graph convolutional neural networks. The structured dataset is transformed into a graph by taking into consideration how frequent a word is for an attribute and the co-occurrence of words inside the same attribute. Once token embeddings are generated, they can be com-

pared against the embeddings of the paired record. Huang et al. (2023) compares each token embedding against the most similar token embedding from the other record and generates similarity (with scalar product) and difference vectors (with element-wise difference). Similarly, Fu et al. (2021) uses attention mechanisms to find the most relevant token to compare against and performs element-wise difference for comparing. Instead of comparing word embeddings against each other, Li et al. (2020a) compares the word embeddings of an instance against a representation of the other record's attribute. Attention mechanisms are used to generate the attribute representations and comparison is done with an element-wise difference. For every approach, after the token embeddings have been compared, they are condensed together in a unique representation for the record using an NN. The representation of each record is concatenated together and fed to the entity-matching model.

6.3.3.2 Attribute-level comparison When records are compared at the attribute level, embeddings must be generated for each attribute. Mudgal et al. (2018), Ebraheem et al. (2018), Kasai et al. (2019) generate word embeddings for each word inside an attribute, then create an attribute embedding by merging the word embeddings together using variants of recurrent neural networks (e.g., LSTM (Hochreiter and Schmidhuber 1997)) or any other method (e.g., summation). Nie et al. (2019) use attention mechanisms to strategically combine the records' token embeddings. Bogatu et al. (2021); Lattar et al. (2020) use sentence encoders such as the universal sentence encoder (Cer et al. 2018) to directly output embeddings for attributes. Once the attribute embeddings are generated for both records of a pair, they can be compared using cosine distance (Lattar et al. 2020), or element-wise difference (Mudgal et al. 2018; Ebraheem et al. 2018; Kasai et al. 2019). For every approach, the result of the comparison for each attribute is concatenated into a vector and fed to the last layers of the model for conducting the prediction task.

6.3.3.3 Record-level comparison Wang et al. (2022) generates record embeddings using a transformers' encoder. Tabular data is transformed into text using the serialization method of Ditto (Li et al. 2020b). The embeddings are compared using element-wise differences and fed to the model for the final prediction task. Because important information might be lost in the comparison vector (e.g., what the records have in common), they also give to the model the embedding of the concatenated records.

6.3.4 Learned comparison

The third category groups all entity-matching approaches that do not have a comparison bias and lets the model learn how to compare instances by itself. These approaches generally adopt superior modeling techniques, such as attention. Mudgal et al. (2018) designed a NN with attention mechanisms and trained the model for entity-matching tasks. Zhao and He (2019) follows a three-step process to detect records referring to the same real-world entity. First, a model predicts the semantic type (e.g., company name, person name) for each attribute of a record. Second, an

entity-matching model trained specifically for the semantic type predicted in the previous step is selected and predicts whether the attribute's value of both records refers to the same real-world entity (e.g., "Bill Gates" versus "William Gates"). The binary predictions (which are generated for each attribute) are fed to a one-layer NN to predict if the records are a match. As discussed in Sect. 6.1.3, the authors in Narayan et al. (2022) evaluate LLMs on data cleaning tasks, notably entity-matching. To perform entity-matching, they first transform the tabular records into text format (see Sect. 6.1.3 for more details on the serialization procedure), show the serialized record to the model, then prompt it to predict whether the records refer to the same real-world entity or not. Li et al. (2020b), Brunner and Stockinger (2020) added a classification layer on top of a pre-trained transformer model and fine-tuned it for the entity matching problem. Similarly to the feature cleaning approaches with transformers (see Sect. 6.1.3), both approaches transform tabular data into text format by concatenating the values of a record. Additionally, Li et al. (2020b) adds special tokens to differentiate the columns' names from the values of a record. For example, for a record of a person living in Montreal, Canada, a serialized entry could look like the following: [ATTRIBUTE NAME] city [ATTRIBUTE VALUE] Montreal [ATTRIBUTE NAME] country [ATTRIBUTE VALUE] Canada. In addition, authors added special tokens in the string for special types (e.g., phone number, zip code).

6.3.5 Other approaches

In this section, we cover other entity-matching approaches that are not included in the previous categories.

- Wu et al. (2020): The authors propose an unsupervised approach for entity matching (i.e., no training data is required). Essentially, two Gaussian distributions are fit to the data, one to the positive pairs, and the other, to the negative pairs. The Gaussian distribution can then be used to estimate the likelihood of a pair of a record being a match or not. They propose optimizations to (1) reduce the number of parameters of the Gaussian distributions, (2) improve the prediction accuracy of their approach, and (3) address the performance issues caused by features with zero variance (i.e., the singularity problem in pattern recognition (Bishop and Nasrabadi 2006)).
- Gottapu et al. (2016): Instead of predicting if two records refer to the same entity, they predict to which entity each record refers. In other words, each class in their approach corresponds to a real-world entity. To classify records, they transformed each record into a sequence of word embeddings and then fed it to a convolutional NN for classification.

6.3.6 Common techniques to improve performance

In this section, we cover various techniques commonly used to improve the entity-matching approaches' performance. Namely, these are data augmentation, semi-supervised techniques, active learning, and transfer learning.

6.3.6.1 Data augmentation Because entity matching is an unbalanced problem (i.e., there are more records that do not refer to the same real-world entity than do), generating negative pairs is fairly easy. Two records randomly selected from a dataset will most likely not refer to the same real-world entity. Wang et al. (2022) randomly select two records that are similar but are not a match and assign them the negative label. The authors argue that selecting similar instances is superior to random selection since it makes the prediction problem more difficult for the model. Generating positive pairs is, however, more difficult than generating negative pairs. Wang et al. (2022) applies task-specific data transformations that preserve the label of a pair of records. Wang et al. (2022) picks any two records whose embeddings have a cosine distance inferior to a fixed threshold. We describe in Sect. 6.3.4 how the embeddings are generated. Instead of selecting instances, Li et al. (2020b), Huang et al. (2023) apply transformations to a record that should preserve the record's identity (e.g., deleting an attribute's value). The modified record can replace the initial record in an already existing pair of records (Li et al. 2020b), or the modified record and the initial record can become a new pair (Huang et al. 2023). To minimize the risk that the pair is not truly a match, Li et al. (2020b) proposes MixDA, a technique inspired by MixUp (Zhang et al. 2017). The approach interpolates the initial record embedding with its transformed version and adds the interpolated version to the training dataset.

6.3.6.2 Semi-supervised techniques To create positive pairs, Kasai et al. (2019) assigns a positive label to any pair that is rated as very likely to be a positive pair according to the entity-matching model. The model is then re-trained on these newly labeled samples and the process can be repeated. Huang et al. (2023) used adversarial learning to generate additional training samples. The goal of the generator model is to fool the discriminator model (i.e., the entity-matching model) into believing that a generated record refers to the same real-world entity as a real record.

6.3.6.3 Active learning Kasai et al. (2019) observed that entity matching is an imbalanced problem (i.e., there exist more negative pairs than positive pairs), and if it is not considered in the active learning process, more negative pairs than positive pairs might be labeled. Hence, the approach tries to send to the oracle an equal amount of positive and negative pairs by referring to the prediction of the entity-matching model, so only uncertain pairs are sent (i.e., when the probability that a pair is a match is close to 0.5). The oracle receives an equal amount of pairs slightly under and over the threshold (of 0.5). Li et al. (2020b)'s approach prioritizes samples with the highest informativeness. They start by grouping unlabeled pairs into clusters based on a custom similarity measure. A pair's informativeness is based on the number of samples that will be labeled if the current pair is labeled positively or negatively. Meduri et al. (2020) benchmarked different active-learning-based approaches for entity matching. They tried different combinations of models (i.e., linear, non-convex, tree-based, and rule-learning) with different acquisition functions to select pairs (i.e., query-by-committee (Freund et al. 1997), how close a sample is from the decision boundary, and likely false positives/negatives (Qian et al. 2017)). They adapt blocking (see Sect. 6.3.1) to sample selection in active learning to fasten the process.

6.3.6.4 Transfer learning Transfer learning refers to reusing a model trained on a different but related task hoping some of the knowledge will be transferable for the task at hand (Wikipedia 2023e). To have better performances, some entity-matching approaches reuse the models trained for another entity-matching problem for the task at hand. For example, a model used to match citation records between two databases could be used for two other databases with different schema (Kasai et al. 2019). Here, we discuss the techniques used to make the transfer of knowledge learned from one task to another more efficient. As discussed in Sect. 6.3.3, Kasai et al. (2019) predicts if two records are a match by comparing the records' tuple embedding. In order to maximize the benefits of transfer learning, they trained the model generating tuple embeddings to be dataset-invariant. In other words, the goal is for the embeddings from the target domain to be similar to the ones of the source domain. The authors use adversarial learning to train the embedding model to generate dataset-invariant embeddings. The role of the discriminator model in the adversarial setting is to predict the provenance of a record (i.e., to predict whether a record comes from the source or target domain). Similarly, Jin et al. (2021) tries to increase domain adaptation of their approach (i.e., transfer learning) by minimizing the KL divergence between embeddings from the source domain with the ones from the target domain.

6.3.7 Evaluation metrics

Similar to feature cleaning and label cleaning, entity matching can be evaluated like a classification problem. Hence, metrics such as accuracy, F1 score, precision, and recall are commonly used (Huang et al. 2023; Li et al. 2020a; Gottapu et al. 2016; Wu et al. 2020; Fu et al. 2021). The execution time of proposed approaches is sometimes presented, e.g., (Wu et al. 2020; Mudgal et al. 2018; Li et al. 2020b).

6.3.8 Comparison of the approaches

As for the feature cleaning approaches, the most performant entity matching methods use pre-trained transformer models. Latent space comparison approaches use these models to generate meaningful embeddings (Wang et al. 2022; Huang et al. 2023), while learned comparison approaches use them to perform entity matching directly (Li et al. 2020b; Narayan et al. 2022). In the work of Brunner and Stockinger (2020), the authors experimented with various transformer architectures and compared their performance to “classical” DL methods in entity matching. On the most challenging datasets, the transformer models outperformed the other approaches by an average of 27.5% for F1 score. Later works leveraging transformer models achieved even better results on various datasets (Narayan et al. 2022; Li et al. 2020b; Huang et al. 2023). In addition to being more performant, these approaches require fewer training samples. Previous DL methods for entity matching require large amounts of samples, which are often unavailable in realistic scenarios (Kasai et al. 2019). In comparison, the prompting approach described in Narayan et al. (2022) surpassed “classical” methods for entity matching with only 10 training samples. A similar work which focuses on maintaining good performances while consuming fewer training samples is presented in Wang et al. (2022). By requiring fewer training samples to

learn to do a task, training is faster for such approaches, sometimes up to the order of a magnitude (Brunner and Stockinger 2020).

While approaches leveraging transformer models tend to surpass others in terms of performance, they can be outperformed by simpler approaches. The authors in Wang et al. (2020) presented a token-comparison approach that obtained better results on selected datasets than other approaches leveraging transformer models (Huang et al. 2023; Li et al. 2020b). As discussed in Sect. 6.3.2, token-comparison approaches compare records by extracting a set of unique and shared terms between the compared records before feeding them to an entity matching model. To motivate their choice of following this approach, the authors highlight that embeddings generated by neural architectures tend to smooth out the differences between records that are vital for entity matching. Hence, for example, the values “6-foot sandwich” and “12-foot sandwich” could be incorrectly detected as the same entity because they are semantically close and have similar embeddings. This strength of token-comparison approaches can also be a double-edged sword, as the authors in (Huang et al. 2023) highlighted. Real-world entity matching datasets are dirty: data can be mistyped or misplaced in an incorrect table column (Mudgal et al. 2018). Thus, finding matching tuples by comparing whether the tokens between two records are exactly the same is error-prone.

Entity Matching: Summary and Future Directions

General existing approaches: The approaches can be grouped based on how the records are compared (for the purpose of entity matching). We identified the following three categories.

- *Token comparison:* A set of common and different words between two records are generated and fed to an ML model that will predict whether the records match or not.
- *Latent space comparison:* The records are compared in a latent space using a distance measure such as cosine distance. The distance measure can then be used by an ML model to predict whether the records match or not. The embeddings used for comparison can be token, attribute, or record embeddings.
- *Learned comparison:* A model ingests both records and predicts whether the records match or not. Contrary to other approaches, the model is entirely responsible for learning how to compare records.

Future Directions:

- Explore how comparing records using different latent representations at the same time can improve entity matching. In Sect. 6.3.3, we described three types of latent-space representation approaches (token-level, attribute-level, and record-level). Combining all three representation approaches could enable the detection of low-level differences between records as well as high-level ones, which could improve performances.
-

6.4 Outlier detection

In Sect. 6.2.4, we presented approaches to detect mislabels using ideas from outlier detection. Here, we cover approaches to detect outliers in a dataset (not for the purpose of detecting mislabels). Borrowed from Ilyas and Chu (2019), we identified three categories of outlier detection approaches: statistic-based, distance-based, and model-based approaches. Note that an approach may belong to more than one

category. For example, the approach in Terrades et al. (2022) uses a model to detect outliers (model-based), but also preprocess data before feeding it to the model using distance-based approaches. We describe how outlier detection tasks are evaluated in Sect. 6.4.5.

6.4.1 Statistic-based approaches

Statistic-based approaches detect outliers by looking for records that are in low-probability regions according to a stochastic model (Ilyas and Chu 2019). To detect outliers, the authors in Pit-Claudel et al. (2016) model a dataset using generative models and use them to detect outliers. Any record with a low probability according to the generative models is marked as an outlier. To improve the performance of their approach, the records are enriched with metadata (e.g., the string length of an attribute).

6.4.2 Distance-based approaches

Distance-based approaches measure the distance between a record and other records to detect outliers Ilyas and Chu (2019). For example, a record that is far from any other data points could be declared an outlier. Guan et al. (2016) compares a record against its neighbors to determine if it is an outlier. If a significant portion of its neighbors come from another class, the record might be an outlier. The authors argue that previous works often incorrectly detect legitimate records from a class with a low number of records as outliers since the records are often in regions with a lot of records from other classes. To mitigate this problem, a weighted average of the labels in the neighborhood of a record is performed to detect outliers and give a larger weight to samples that are close and that are from the minority class. Similarly, Terrades et al. (2022) uses the neighbor of a sample to determine if it is an outlier. The authors make the observation that outliers often are in clusters of samples with high heterogeneity of classes, or in clusters where a large proportion of the samples have the same class. Thus, to detect outliers, both aspects are measured and fed to a model for prediction.

6.4.3 Model-based approaches

Model-based approaches primarily rely on a classifier to detect outliers Ilyas and Chu (2019). Detecting outliers is formulated as a binary prediction task. Liu et al. (2019) proposes an approach to detect outliers that is inspired by the GAN architecture (Goodfellow et al. 2014). Their approach consists of a discriminator model and k generator models whose goal is to generate synthetic samples. Each generator is trained on a different subset of similar data points to increase the diversity of generated samples. Once trained, the discriminator model can be used to detect outliers. The authors in Jiang et al. (2023) use the reconstruction error of an autoencoder (similar to autoencoder-based feature cleaning, see Sect. 6.1.4) and the distance between a record and the closest class distribution to detect outliers (similar to

distance-based approaches, see Sect. 6.4.2). They conducted experiments with two distance measures, namely Mahalanobis and Euclidean distance. The more a record is poorly reconstructed and far from any class distribution, the more likely it is an outlier.

6.4.4 Other approaches

In this section, we cover outlier detection approaches that were not included in the previous categories.

- Papastefanopoulos et al. (2021): Several unsupervised detecting approaches are proposed to detect outliers; however, according to the authors they rely predominantly on a single metric (e.g., distance, density, etc.), and each has its shortcomings. In order to address this issue, a voting ensemble-based approach of multiple outlier detection methods relying on different metrics is introduced. To improve performances, non-informative methods are filtered from the ensemble using an unsupervised spectral feature selection algorithm (Zhao and Liu 2007).
- Domingues et al. (2018): The authors conduct a study in which they compare outlier detection methods. The compared methods are grouped into probabilistic-based [e.g., robust kernel density estimators (Kim and Scott 2012)], neighbors-based [e.g., Subspace outlier detection (Kriegel et al. 2009)], distance-based [e.g., Mahalanobis distance (Ben-Gal 2005)], information theory-based (e.g., Kullback–Leibler divergence (Filippone and Sanguinetti 2010)), NN-based [e.g., Grow When Required (GWR) network (Marsland et al. 2002)], domain-based methods [e.g., One-class SVM (Schölkopf et al. 1999)] and isolation methods [e.g., Isolation forest (Liu et al. 2008)]. The study shows that isolation forest (Liu et al. 2008) is an efficient method for detecting outliers in terms of average precision, training and prediction time, and scalability.
- Alimohammadi and Chen (2022): Similar to Domingues et al. (2018), the work focuses on evaluating techniques for detecting outliers in time-series data. The evaluated techniques include ML-based techniques [e.g., one-class SVM (Schölkopf et al. 1999)], statistical-based techniques (e.g., z-score (Rosner 1983)) and regression-based techniques (e.g., polynomial fit (Motulsky and Brown 2006)). The experiments show that KNN is the optimal choice in terms of ML performance, simplicity of configuration, and computation time.

6.4.5 Evaluation metrics

Similar to feature cleaning, label cleaning, and entity matching, outlier detection can be evaluated like a classification problem. However, different metrics tend to be used to measure performance. It is common for outlier detection approaches to detect outliers by applying a threshold over a numerical value. Thus, to factor out

the threshold configuration, metrics such as “Area Under the Receiver Operating Characteristic” (AUROC) (Jiang et al. 2023; Papastefanopoulos et al. 2021; Terrades et al. 2022) are commonly used. The execution time (Alimohammadi and Chen 2022; Domingues et al. 2018) and memory consumption (Domingues et al. 2018) of the proposed approaches are sometimes presented.

Outlier Detection: Summary

General existing approaches:

- *Statistic-based approaches*: Records are marked as outliers if they are in a low probability region according to a probability model.
 - *Distance-based approaches*: The distance between a record and others is measured to determine if the record is an outlier.
 - *Model-based approaches*: A model plays a central part in determining if a record is an outlier.
-

6.5 Imputation

In this section, we cover approaches to impute values in datasets with missing values. Abidin et al. (2018) compares the performance of three ML algorithms (i.e., decision trees, k-nearest neighbor, and Bayesian networks) for imputation on 10 datasets. They found that Bayesian networks are the most accurate for most datasets, but they are expensive to train. Razavi-Far et al. (2020) proposes two techniques, kEMI and kEMI+ to impute categorical and numerical missing data. Both techniques first utilize the K-Nearest Neighbors (KNN) algorithm as a local search algorithm to find the K-top similar records to a record with missing values. Then kEMI invokes the Expectation-Maximization Imputation (EMI) algorithm which uses the feature correlation among the K-top similar records to impute missing values. The idea is to use conditional probability distribution of a feature given the other features using only the K-top similar records. In contrast, kEMI+ repeatedly invokes EMI to gather a collection of estimates for the missing value and combines them using Dempster-Shafer fusion (AP 1967; Dempster et al. 2008). Although kEMI+ can be more accurate than kEMI, it is less scalable. A novel technique proposed in (Silva-Ramírez and Cabrera-Sánchez 2021) that combines Adaptive Resonance Theory (ART) algorithm (Carpenter et al. 1991) with Co-active Neuro-Fuzzy Inference System (CANFIS) (Tfwala et al. 2013) to impute missing data. CANFIS integrates the reasoning capabilities of fuzzy systems and the computational capabilities of NN such that the fuzzy logic provides a linguistic representation of variables and performs fuzzy inference, while the NN performs computations on the fuzzy rule outputs and maps them to crisp output values. ART complements CANFIS by providing unsupervised learning and clustering of the input data which helps the fuzzy rules construction and enhances the model representation capabilities.

Table 3 Summary of the challenges mentioned in Sect. 7

Sectiona	Challenges and problems
7.1	For many data cleaning activities, finding a labeled dataset is a challenging task, since they tend to be scarce and are rarely centralized in one place (e.g. with the use of a dataset repository)
7.2	Most data cleaning tasks are imbalanced problems: samples tend to be clean rather than dirty. As a result, labeled data cleaning datasets have only a few dirty (labeled) samples
7.3	We have not observed any actively maintained tool to allow practitioners to leverage the latest advances in data cleaning
7.4	Real-world datasets may have more than one type of error. However, the vast majority of data cleaning approaches covered in this study clean only one type of error. Additionally, there is little knowledge regarding how cleaning one type of error impacts how effectively other data errors can be cleaned

Imputation: Summary

General existing approaches: The approaches covered in our study impute missing data by using feature correlation or by combining NNs with fuzzy systems.

6.6 Holistic data cleaning

In this section, we present data cleaning approaches that try to clean more than one type of error at a time.

6.6.1 Data cleaning pipeline generation

Instead of directly cleaning data, the approaches in this category try to find the optimal sequence and configuration of data cleaning tools. The process of building the optimal cleaning pipeline can be formulated as a search problem. Given a metric to optimize, the goal is to find the optimal ordering and configuration of cleaning tools. These kinds of approaches are sometimes referred to as a cleaning optimizer (Neutatz et al. 2021). The main factor that differentiates approaches from one another is the objective function and the optimization algorithm used, which we describe below.

Optimally, one would want to decrease the total number of errors in the dataset. However, that information is not always known a priori. Thus, a common approach when cleaning for ML is to use the performance of the model trained on the cleaned dataset as a proxy of data quality (Berti-Equille 2019; Krishnan and Wu 2019). If the ultimate goal of cleaning the dataset is to improve the model's performance, this objective can be preferred to any other proxy of dataset quality. Li et al. (2019) showed that not all data cleaning tools improve a model's performance. Thus, cleaning a dataset using ML performance as a metric to maximize can avoid cleaning operations that decrease model performance. Other metrics can be used as well. For

example, Krishnan and Wu (2019)'s approach can minimize the number of outliers in a dataset or the number of integrity constraints violated.

Technically, ML hyperparameter optimizers, such as Python Hyperopt (Bergstra et al. 2013), could be used to configure the data pipelines. However, as Krishnan and Wu (2019) pointed out, these optimizers do not leverage the incremental nature of data cleaning. Effectively, instead of evaluating the impact of a data cleaning tool on a model's ML performance right after it has been applied to a dataset, ML optimizers evaluate complete solutions (i.e., a fully configured data cleaning pipeline). To address that problem, Krishnan and Wu (2019) uses beam search to find optimal pipeline configurations. The least promising pipelines are periodically pruned out based on the performance of the model trained on the current dataset. Another strategy proposed by Berti-Equille (2019) is to use Reinforcement Learning (RL) to find the optimal configuration for data-cleaning pipelines. In their work, an agent iteratively selects a data cleaning tool that will be applied to a dataset. The change in a model's performance after applying the data cleaning tool serves as the reward function of the agent, and the state is the last cleaning tool applied to the dataset. In addition to the data cleaning tools, pre-processing operations, such as normalization, can also be selected by the agent.

Instead of searching for new pipeline configurations tailored to a dataset, Gemp et al. (2017) reuses data cleaning pipelines that were shown to be effective on similar datasets. For this to be effective, a good measure of similarity between datasets must be used. The authors chose to represent datasets by vectors of 22 dimensions composed of common meta-features from the literature, such as mean feature skew. The datasets' vectors are compared against one another using L1 distance. While the results were inconclusive, they argue that their approach could open the path to new ways of generating data-cleaning pipelines.

6.6.2 Other approach

Li et al. (2019) benchmark different data cleaning techniques on real-world datasets and observe their impact on the performance of ML classification tasks. To perform their experiments, they used data cleaning tools from all the data cleaning categories discussed in this review (i.e., Sects. 6.1 to 6.5). Using different datasets, the authors measured the impact of using each tool individually on ML performance, then they repeated the experiments with different combinations of tools. Some of the findings of the study are that an improvement to a model's ML performance because of data cleaning will generalize to other models and that no single data cleaning tool is the best for all datasets.

Holistic Data Cleaning: Summary and Research Opportunities

General existing approaches:

- *Data cleaning pipeline generation*: Instead of directly cleaning data, these approaches try to find the best sequence and configuration of data cleaning tools to optimize a metric (e.g. ML accuracy of a model trained on the cleaned dataset).

Future directions: We elaborate on future research directions for holistic data cleaning in Sect. 8.5.

7 Challenges in DC & ML

By conducting this review, we observed challenges spanning multiple data cleaning activities. In this section, we have a discussion about them hoping to help the research community address some of the most important ones. We provide in Table 3 a simplified description of the challenges listed here. As a result of this effort, we propose future work directions in Sect. 8.

7.1 Scarcity of data cleaning datasets

To evaluate a data cleaning approach or train an ML model to be used in a data cleaning approach, one needs a labeled dataset. As building a labeled dataset is a laborious process (Gauen et al. 2017), researchers are interested in using the existing public datasets. Additionally, using public datasets provides a baseline for researchers to compare their approaches. The release of large public datasets has been known to propel research in ML (Gauen et al. 2017). For instance, the release of ImageNet datasets (Deng et al. 2009) led to the birth of new neural architectures such as AlexNet (Krizhevsky et al. 2017) and VGG (Simonyan and Zisserman 2014). For data cleaning tasks, there exist popular datasets used across many papers such as the Hospital dataset, the Flights dataset, and the Beers dataset (all of which are described in Mahdavi et al. (2019)). However, for researchers, finding and selecting a dataset to evaluate their data cleaning approaches can be a laborious process as it entails searching through many data cleaning papers for datasets. Furthermore, because data cleaning papers sometimes forget to provide the source of a dataset they used (and instead reference the paper they extracted the dataset from), obtaining the actual dataset can be arduous.

7.2 Imbalanced datasets

In addition to the lack of public labeled datasets (Sect. 7.1), the latter often only have a limited number of labeled dirty examples. As explained in Sects. 6.1.8 and 6.3.6, datasets of data cleaning activities are often imbalanced. The datasets used for feature cleaning generally only have a few dirty (labeled) records (Nashaat et al. 2021; Heidari et al. 2019). Indeed, when cleaning the features of a dataset, one can expect to find a larger number of values that are correct rather than not. Hence, the cleaned versions of datasets have only a few labeled dirty samples. Similarly, for

Table 4 Summary of future work recommendations from Sect. 8

Sections	Research opportunities
8.2	Develop approaches to find an optimal parametrization of data augmentation techniques Develop approaches to automatically filter out invalid augmented samples
8.1	Create datasets repositories for all data cleaning activities (e.g., feature cleaning) Build a taxonomy of data errors Publish data cleaning datasets that are labeled Facilitate the process of correcting errors in public ML datasets and keep track of the changes
8.3	When proposing an approach, a researcher should implement its approach in a public data cleaning library. This should be enforced by publishers' reviewers
8.4	Develop new data-cleaning pipeline generation approaches Develop approaches that can clean more than one type of error at a time Share LLMs pre-trained on data cleaning tasks
8.5	Explore how LLMs can be leveraged for data cleaning tasks Explore how tabular data can be transformed into text for data cleaning with LLMs Develop tools combining data cleaning with other data pre-processing activities Analyze how different data cleaning activities affect one another when applied to the same dataset
8.6	Develop interactive data cleaning approaches

entity matching, most records in a dataset do not refer to the same real-world entity. Hence, generating "negative" data is easier than "positive" data since any pair of records sampled from a dataset is likely not to refer to the same real-world entity (Kasai et al. 2019).

7.3 Lack of proper tooling

Data cleaning is known to be tedious and time-consuming (Sambasivan et al. 2021). Researchers have reported that data cleaning can take up to 80% of data scientists' time (Press 2022). However, as highlighted in Sambasivan et al. (2021), data cleaning is one of AI's most under-valued and de-glamorized aspects. To clean data efficiently, practitioners might look for data-cleaning tools. If the practitioner wants the latest advances in data cleaning, he may look for the studies' replication packages. However, not all papers provide a replication package. Out of the 101 papers included in our study, only 20 provided one. Additionally, using replication packages is not convenient for practitioners. They must familiarize themselves with the application programming interfaces (API), which might not be user-friendly and differ from one replication package to another. Additionally, setting up the tool (e.g., installing dependencies) can be tedious and documentation may be lacking. Thus, to address this issue, Müller et al. (2021) proposes OpenClean, an open-source library where all data-cleaning approaches can be centralized and made available via a unique API. Similar initiatives have gained a lot of traction in ML, such as

Scikit-learn (Pedregosa et al. 2011). On the contrary, OpenClean has not received any support apart from its creators two years ago. Thus, a practitioner looking for data-cleaning tools might be deterred from using this open-source library.

7.4 Siloed data cleaning approaches

Except for the papers in Sect. 6.6, the papers included in our study considered each data cleaning task individually. However, in the real world, practitioners find themselves cleaning more than one type of error in a dataset. For example, a practitioner could have to remove outliers, and then clean the features in a dataset. Cleaning each data error individually could lead to inefficiencies. For example, Li et al. (2019) has shown that cleaning all data error types in a dataset does not necessarily improve a model's performance. Furthermore, cleaning all types of errors can lead to worse performance than cleaning only one type from a dataset. While understanding the interactions between the data cleaning approaches is important to effectively clean a dataset, they are not fully understood yet. The current approaches that clean more than one type of error in a dataset (Sect. 6.6) do so by following the predictions of an ML model (Berti-Equille 2019) or by adopting a beam-search approach Krishnan and Wu (2019). They do not understand the influence the underlying data cleaning tools have on each other's performance and rather follow a search strategy guided by the impact the data cleaning tools have on a proxy of data quality.

8 Future directions

In this section, we propose future work directions based on the challenges mentioned in Sect. 7 and the opportunities we have discovered in our study. Comparatively to the future directions proposed in Sect. 6 for each category in our taxonomy, the suggestions made in this section are applicable to all data cleaning tasks and do not stem from the idiosyncrasies of the respective problems. They are based on the challenges and opportunities shared between the tasks. We provide a summary of the research directions proposed in this section in Table 4.

8.1 Data cleaning datasets

To evaluate a data cleaning approach or train an ML model to be used in a data cleaning approach, one needs a labeled dataset. However, as mentioned in Sect. 7.1, finding publicly available labeled data cleaning datasets can be a laborious task since there is not a central repository of such datasets for most data cleaning tasks. Previous work has led to the birth of a dataset repository for entity matching (Das et al. 2016). However, no studies included in our work mentioned such repositories. Labeled datasets are of paramount importance since they provide a benchmark to compare different approaches in addition to enabling the development of ML

approaches. Hence, a meaningful contribution to data cleaning would be to develop a dataset repository for the various data cleaning tasks covered in this study.

Providing a data repository would open up other new research directions. For feature cleaning, one could build a taxonomy of errors in ML datasets using the dataset repository. This taxonomy would help researchers evaluate their feature-cleaning approach in a systematic way. Instead of solely selecting datasets based on previous works, the researchers would be able to select datasets that have the error types their approach tries to address.

Finally, in the same effort to provide data-cleaning datasets to researchers, future works could target publishing labeled data-cleaning datasets. To address the lack of labeled datasets, previous works (Flokas et al. 2022; Liu et al. 2022; Neutatz et al. 2019) have used datasets with artificial errors (i.e., errors injected into clean data). Thus, in an attempt to enable researchers to evaluate their approach against real-world errors, we encourage future works to publish labeled data-cleaning datasets. Additionally, maintainers of any public ML dataset could facilitate the process of identifying data errors and repairs, so as to organically create labeled data-cleaning datasets.

8.2 Data augmentation

In Sect. 7.2, we described the issue plaguing many data cleaning problems: class imbalance. Indeed, labeled data cleaning datasets tend to have more clean (labeled) data than dirty (labeled) data. Hence, it is common for data cleaning techniques to leverage data augmentation to generate more training samples for the class that is underrepresented. While it is true that data augmentation can improve a model's performance, it can also reduce it, if not properly tuned (Miao et al. 2021; Wei and Zou 2019). For example, a data augmentation operator could transform a record in a way that its initial label does not apply anymore. Thus, when using data augmentation strategies, it is important to ensure they are correctly parameterized. Past studies in computer vision have tried to design search algorithms to automatically find the best parametrization of data augmentation techniques (Cubuk et al. 2019; Lim et al. 2019). Future works could continue in this direction and evaluate their approach for data cleaning tasks. While a good parametrization increases the chances that good samples are generated from the data augmentation tool, all samples are not guaranteed to be valid. Miao et al. (2021) proposes an approach to automatically filter out augmented samples that are invalid. Additionally, the approach automatically weights examples based on their likeliness to improve a model's performance. Thus, the model learns more from samples that are weighted more. While not a complete replacement for a good parametrization of data augmentation tools, this strategy makes configuring data augmentation tools less critical. We encourage researchers to develop approaches to automatically filter out augmented records that hinder a model's performance.

8.3 Tooling

Data cleaning is known to be tedious and time-consuming (Sambasivan et al. 2021). Researchers have reported that data cleaning can take up to 80% of data scientists' time (Press 2022). However, as highlighted in Sambasivan et al. (2021), data cleaning is one of AI's most under-valued and de-glamorized aspects. To clean data efficiently, practitioners might look for data-cleaning tools. However, as illustrated in Sect. 7.3, we only discovered one data cleaning tool openly accessible to practitioners. Only a few papers provided replication packages, which are not a convenient alternative to a public library. Despite being an efficient vehicle to make research developments available to practitioners faster, the open-source library we discovered has not received any contribution for the last 2 years. Hence, we encourage future works on data cleaning to provide a ready-to-use implementation of their approach on open-source libraries such as OpenClean (Müller et al. 2021). Additionally, we suggest considering contributions to open-source libraries when evaluating data-cleaning approaches so as to incentivize researchers to contribute.

8.4 Large language models

We presented in Sects. 6.1.3 and 6.3.4 data cleaning techniques that used LLMs to clean data. These approaches generally obtained state-of-the-art results on their respective tasks. Recently, OpenAI released ChatGPT,¹⁰ an LLM which has shown impressive performances on diverse tasks, from solving programming bugs (Surameery and Shakor 2023) to passing the Bar exam (CNE 2023). We believe that future work should continue proposing data cleaning approaches that leverage LLMs, so as to benefit from the improvements with these models and achieve state-of-the-art performances (Narayan et al. 2022).

As shown in Sects. 6.1.3 and 6.3.4, Narayan et al. (2022) ask questions in natural language to an LLM in order to clean data. For example, Narayan et al. (2022) uses the following sentence to clean data: "Is there an error in attribute X" (where X is the name of an attribute), to which the model answers "yes" or "no". Before asking the question, the tabular record is presented to the model in a special format that we describe in Sects. 6.1.3 and 6.3.4. Their approach achieved state-of-the-art performances. We believe that an interesting future research direction would be to explore and compare different ways tabular data can be formatted for data cleaning with LLMs. Similarly, we encourage researchers to explore and compare different ways to prompt an LLM for data-cleaning tasks. This could build upon the current knowledge in prompt engineering (White et al. 2023).

Finally, we encourage future works to train and share LLMs trained on data cleaning tasks. Other researchers and practitioners could use these LLM models for their data-cleaning problems. A pre-trained model could leverage the knowledge it gained from previous data cleaning tasks for any other one. Similar contributions have been made in other fields (Beltagy et al. 2019; Araci 2019). For example, in

¹⁰ <https://openai.com/blog/chatgpt>.

finance, Araci (2019) developed FinBert, a language model fine-tuned on finance texts.

8.5 Holistic data cleaning

In Sect. 6.6, we presented two approaches to perform holistic data cleaning (Berti-Equille 2019; Krishnan and Wu 2019). These approaches search for the best order and configuration of data cleaning tools. They propose an attractive solution to practitioners who must configure various tools to clean diverse types of errors in datasets. However, only 2 papers included in this study proposed an approach to address that situation. Thus, we believe there is space in the literature to propose other data cleaning pipeline generation (DCPG) approaches. Future works could build upon the work done in AutoML (He et al. 2021a), since the latter shares many similarities with DCPG. Indeed, DCPG could be seen as a subset of AutoML, where instead of considering the whole ML pipeline (as with AutoML), the search algorithm only focuses on a specific step in the pipeline (data cleaning).

More broadly, we encourage researchers to develop tools that try to clean more than one type of error in a dataset. A previous study (Wang et al. 2022) has demonstrated how two different types of errors (i.e., duplicates and feature errors) could be cleaned following a similar strategy. The authors (Wang et al. 2022) were able to adapt their entity-matching approach for feature cleaning with only minor modifications. Potentially, other data cleaning activities share similarities that are still uncovered. Thus, designing a learning approach that cleans more than one type of error at a time could clean a dataset more effectively than using each type of tool independently.

On an even broader scale, future work may concentrate on developing tools combining data cleaning with other data pre-processing activities. For example, in Sect. 6.6, we presented the work of Berti-Equille (2019), who combined other data preparation tasks (e.g., feature normalization and selection) with data cleaning techniques (e.g., outlier detection). Tae et al. (2019) proposed a framework that combines data cleaning with unfairness mitigation and poisoned sample removal.

To effectively design tools to holistically clean a dataset, one must understand how these tools interact with one another. However, we found little knowledge about that in our review of the literature, as mentioned in Sect. 7.4. As we covered in Sect. 6.6, Li et al. (2019) studied how different types and combinations of data cleaning tools impacted the performance of a model trained on the cleaned dataset. Instead of only considering an ML model's performance, researchers could study how applying one type of data-cleaning activity affects the performance of another one. For example, it could be interesting to study how feature cleaning influences an entity-matching model. Maybe cleaning data errors would make entity matching easier. Additionally, future works could also replicate Li et al. (2019)'s study but with a larger set of tools, since Li et al. (2019) used only one tool for some of the data cleaning activities that were considered (e.g., label cleaning). This work could use some of the tools reviewed in this paper.

8.6 Interactive data cleaning

Most of the papers covered in this SLR (92/101 papers) provide standalone data-cleaning solutions that do not need someone's efforts to function (except for dataset labeling). In other words, the approach automatically cleans data once sufficient training data is provided. However, in practice, human experts are often involved in the cleaning process, since it is usually impossible for machines to clean datasets flawlessly (Ilyas and Chu 2019). We refer to cleaning data using an expert's feedback as interactive data cleaning. These experts can provide valuable input to the data-cleaning process by solving various kinds of uncertainties. For example, for feature error repair (see Sect. 6.1), experts can validate the repairs proposed by the approach to ensure that no errors are introduced in the dataset. Considering the low number of interactive data-cleaning approaches covered in this SLR and their relevance in practice, we encourage the community to develop such solutions.

9 Related works

In this section, we present works that are related to our study. In Sect. 9.1, we cover studies that, like ours, review the literature on DC & ML. While, as mentioned in Sect. 3.2, we limited ourselves to data cleaning works for image, text, and tabular data, data cleaning is applied to other data types as well. For instance, in SE, data cleaning is applied to code datasets. Hence, we present, in Sect. 9.2, data cleaning works in SE. In Sect. 9.3, we provide an overview of label-noise robust learning, an alternative method to label cleaning for improving the performance of ML models in the presence of label noise. We also highlight the synergies and similarities between the two approaches.

9.1 Literature reviews in DC & ML

Previous works partially cover DC & ML. The authors in Ilyas and Chu (2019) explain data cleaning and describe different data cleaning activities (e.g., entity matching, outlier detection). The authors reserved a full chapter on ML-based techniques for data cleaning. However, their work is not as exhaustive (in terms of approaches covered) as the work presented in this paper, and has a broader scope: all of data cleaning (not limited to ML). The authors in Neutatz et al. (2021) present their vision of a holistic data-cleaning framework for ML applications. They also provide an overview of recent DC4ML or ML4DC approaches. Contrary to our work, they only provide an overview of the topic and do not examine the different techniques systematically and in detail as we do in this work. Similarly, Ilyas and Rekatsinas (2022) review the relationship between data cleaning and ML (i.e., DC4ML and ML4DC) and present current challenges in data cleaning for ML applications. In addition, they describe robust ML approaches. Their work is different from ours because their goal is not to systematically review the literature. The authors in Thirumuruganathan et al. (2020) explain how DL techniques can

be leveraged for cleaning data and present other studies that leverage DL to clean data. As a vision paper, it proposes some research directions to the community. In Roh et al. (2019), authors conducted a survey of data collection for ML. They argue that data collection is composed of different processes, which feature cleaning, label cleaning, and entity matching are part of. Similar to ours, the data cleaning approaches presented are part of ML4DC or DC4ML. Contrary to our work, their scope is broader and they are not performing a systematic literature review. In the extended version of their study (Whang et al. 2023), in addition to covering data collection for ML, the authors cover robust model training and fair model training. Similar to the two previous works, a survey on data management for ML is reported in Chai et al. (2022) and includes a section to describe data cleaning for ML. Dong and Rekatsinas (2018), Laure et al. (2018), Chu et al. (2016b) are tutorials that briefly discuss how ML can be used for data cleaning.

Other related works focus on one of the data-cleaning activities covered in our review (e.g., feature cleaning, label cleaning, etc.). For example, Christophides et al. (2020) reviews the literature on entity matching including a section on entity matching using ML. Barlaug and Gulla (2021) performs a survey on neural networks for entity matching. Authors in Johnson and Khoshgoftaar (2022), Feng et al. (2023), Karimi et al. (2020) review techniques to handle labeling errors in ML datasets including label cleaning. Authors in Wang et al. (2019), Boukerche et al. (2020) perform a survey of outlier detection techniques and cover ML-based techniques. The works of Lin and Tsai (2020), Adhikari et al. (2022) survey techniques to impute missing data and discuss how ML can be used to impute missing data.

9.2 Data cleaning and software engineering

Datasets are used in various SE tasks, and, like any other dataset, they require quality assurance. Liebchen and Shepperd reviewed data quality within the context of empirical SE (Liebchen and Shepperd 2016). Tasks in empirical SE include vulnerability prediction or software effort estimation. The authors concluded that there is a growing acknowledgment within the community, compared to their prior study in 2008 (Liebchen and Shepperd 2008), regarding the importance of data quality, with increased efforts directed towards exploring techniques for automatically detecting (and sometimes repairing) issues related to noise. They advised researchers to prioritize data quality, as even minor details can significantly impact results obtained from the data. They acknowledged the necessity of establishing mechanisms for assigning quality labels and discontinuing the use of datasets with serious concerns, since using non-qualified data is not productive for researchers or practitioners and wastes resources. Moreover, they highlighted that proposing protocols for describing data and addressing quality issues is gaining traction in the SE community. To ensure trustworthy data-driven research in SE, the authors clarified that having confidence in data quality and minimizing noise is crucial. Finally, they advised collaboration with practitioners in the collection and interpretation of data, as it will ultimately lead to more reliable and actionable research. Bosu and MacDonell proposed a taxonomy of data quality challenges in empirical SE (Bosu and MacDonell 2013).

The authors reviewed the literature on data quality within empirical SE to pinpoint various quality challenges. They categorized these challenges into three primary groups. The first group encompasses data characteristics that render observations unsuitable for model learning (accuracy), like noise, outliers, and inconsistency. The second group includes challenges regarding the applicability of a model to different datasets (relevance), for example, heterogeneity and amount of the data. Lastly, the third group comprises factors restricting data accessibility and trustworthiness (provenance), like the commercial sensitivity of a dataset. Their goal was to utilize this taxonomy to highlight to the broader empirical SE community the challenges inherent in modeling with data and the proposed methodologies for identifying and resolving such challenges.

Bhandari et al. conducted a systematic literature review on data quality issues in software fault prediction (Bhandari et al. 2023). The authors reviewed 145 research papers published prior to November 2021 and investigated 5 aspects: data quality issue, pre-processing technique, modeling technique, dataset, and performance measures. Their results suggested that various data quality concerns, including data dimensionality and class imbalance, have received significant attention in existing SE literature for fault prediction. Nonetheless, issues such as class overlapping and missing data are particularly relevant to fault prediction datasets and warrant additional investigation. The comparative impact of addressing different data quality concerns remains largely unexplored. While C4.5, naive Bayes, multilayer perceptron, support vector machine, and random forest are commonly employed classifiers, their sensitivity to specific data quality issues should be considered when employing them. Accuracy, precision, recall, and area under the curve are frequently used performance metrics in fault prediction. The authors concluded that the presence of data quality issues in fault prediction datasets adversely affects classifier performance, highlighting the need for further research in this area.

In reaction to the previous studies describing the data quality issues of software engineering datasets, other works proposed data cleaning approaches to correct dirty data. A number of these works are dedicated to cleaning software vulnerability datasets. Croft et al. (2023) attempted to understand the characteristics of data quality in software vulnerability datasets. Authors measured the prevalence of each attribute in existing datasets and provided a discussion on the root causes of identified issues. They reported that, in real-world vulnerability datasets, 20–71% of labels were not accurate, leading to a decrease (up to 65%) in the performance of the trained models. Guo and Bettaieb (Guo and Bettaieb 2023) define three critical issues (i.e., data imbalance, low vulnerability coverage, and biased vulnerability distribution) and three secondary issues (i.e., errors in source code, mislabeling, and noisy historical data) that can affect the model performance when trained on software vulnerability datasets. Croft et al. (Croft et al. 2022) discussed the commonly used approaches for dataset cleaning for the software vulnerability prediction task. They include removing irrelevant code files, replacing the user-defined variables and function names with generic tokens, removing comments, blank lines, and non-ASCII characters, and removing duplicate code modules.

Instead of cleaning software vulnerability datasets, Liang et al. (2023) introduced a data cleaning method tailored for the comment updating task, which is a

burgeoning SE endeavor focused on automatically updating comments to reflect changes in source code. Their proposed solution, called CupCleaner (Comment UPdating's CLEANER), is a semantic and overlapping-aware approach. CupCleaner calculates a score by considering both the semantics and overlapping information between the code and comments. By analyzing the score distribution, it identifies and filters out data with low scores located at the tail end of the distribution, effectively removing potentially unclean data. Similarly, the authors in Shi et al. (2022) developed a tool to automatically comment code after having conducted an analysis of noisy data in code comments. They evaluated their approach on 4 code summarization datasets.

While data cleaning can be applied on the datasets of various software engineering tasks, it is also interesting to note that a few of these tasks have similarities with data cleaning. For example, fault localization and fault repair seek to locate and fix errors in code (Zhang et al. 2023a), which is similar to detecting and repairing errors in data (i.e., data cleaning). However, there are some differences between these tasks as well. First of all, artifacts are different: while the focus of data cleaning activities is on the data (in a dataset), fault localization/repair tasks are concentrated on the code. The symptoms of errors in data and code are different as well. Poor quality of the data leads to poor performance (in terms of prediction accuracy) of the ML model trained on it (Braiek and Khomh 2020), however, in software faults we observe compilation/runtime errors, memory issues or software failure (Wong et al. 2016). Moreover, evaluation metrics are sometimes different as we discussed in detail in Sect. 6. Data cleaning activities are usually evaluated as a classification task, i.e., by measuring precision, recall, and F1 score. In some cases, the impact of data cleaning is measured on a downstream ML model. While evaluating fault localization techniques shows similarities with data cleaning (in the precision of identifying the location of errors in code and data), the effectiveness of a program repair is usually evaluated by passing unit tests.

9.3 Label-noise robust learning

Similar to label cleaning, label-noise robust learning approaches try to improve the performance of ML models in the presence of noise. However, in order to achieve that goal, label-noise robust learning approaches modify the learning algorithms instead of fixing data errors. Although data cleaning and robust learning are distinct approaches, they have similarities, as robust learning sometimes involves identifying and mitigating the effects of mislabeled samples in the training model. For example, DL techniques may incorporate a noise layer into the network, which aims to identify potentially incorrect labels based on discrepancies between the model's predictions (derived from prior layers) and the actual labels, thereby fine-tuning the model's predictions and reducing the impact of mislabeled data (Thekumparampil et al. 2018). Additionally, some strategies minimize the influence of mislabels by down-weighting samples likely to be incorrect, by minimizing the loss on clean validation data (Ren et al. 2018). Other effective strategies include employing a teacher-student model (knowledge distillation) (Shi and Wu 2021) or enabling models to abstain from making predictions on ambiguous

samples, thereby allowing the model to concentrate on clearer, more reliable training data Zhang et al. (2023b). Conversely, other robust learning approaches do not focus on the samples but instead adjust hyperparameters, such as the learning rate and regularization parameters (Zhou et al. 2021). Moreover, some classification methods are designed to be used for both; robust learning and data cleaning like the approach proposed by Zhang et al. (Zhang et al. 2018a). This approach employs a separate autoencoder to learn the features of each class and then utilize minimum reconstruction error for identifying mislabeled samples or classifying data points.

The relationship between robust learning and data cleaning is synergetic. As explained in Sect. 6.2.1, robust learning approaches can be used to improve the performance of label cleaning approaches. While data cleaning seeks to directly eliminate or rectify mislabels, robust learning develops models that are inherently more tolerant to such noise. This can be achieved by identifying and compensating for mislabels during the learning process or through adjustments in the training pipeline.

10 Threats to validity

We use the threats to validity described in Feldt and Magazinius (2010), Zhou et al. (2016) to analyze the limitations of our works. We detected three types of threats to validity in our work: construct validity, internal validity, and conclusion validity.

10.1 Construct validity

First, there is a risk that our search query does not allow us to exhaustively cover the field of interest of this study, preventing us from effectively answering our RQs. As mentioned in Sect. 3.4, we did preliminary searches to build a list of keywords that most likely capture the majority of the papers we are interested in. However, there is a risk that we missed keywords that would have led us to papers relevant to our study. To mitigate that threat, we performed snowballing, as described in Sect. 3.6. The second threat relates to the existence of papers relevant to the study but published after our data collection date, and hence, not included in this paper. Future works should consider replicating our study. There is also a risk that the academic databases used in this study might be incomplete, i.e., missing some papers relevant to our study. To mitigate this threat, we searched on Google Scholar, which allowed us to find papers from other databases (which we had access to but were not on our list). Another threat to construct validity relates to the risk that our inclusion and exclusion criteria might have excluded works relevant to our study. However, similar criteria have been used successfully in previous studies (Tamboon et al. 2022). Also, papers relevant to our study might have been excluded because they processed a different data type than image, text, or tabular data (i.e., the data types considered in our study). However, as mentioned in Sect. 3.2, we selected three of the most common data types. Thus, we believe our scope allowed us to cover the most relevant papers for our study.

10.2 Internal validity

There is a risk that papers relevant to our study have been incorrectly excluded from our study while (1) applying inclusion and exclusion criteria (Sect. 3.5.3) or while (2) applying quality control questions (Sect. 3.5.4). To avoid excluding relevant papers, in the first filtering step, we included papers in case of doubt. As mentioned in Sect. 3.5.3, a second researcher reviewed the list to exclude any paper that should not be included. To avoid excluding relevant papers in the second filtering step, a second researcher read every paper excluded because of the quality control questions and verified that the exclusion of a paper is justified, as mentioned in Sect. 3.5.4. There is also a risk that one of the authors misunderstood the approach proposed in a paper leading to an incorrect explanation of the paper in our review. To mitigate this threat, each reviewer asked other reviewers to read the paper and share their understanding whenever they felt they did not fully understand the approach. Furthermore, a researcher verified that the papers' descriptions in our review were coherent with the papers' abstracts.

10.3 External validity

External limitations of our study are related to the generalization of the presented results. We attempted to cover different topics in DC4ML and ML4DC as much as possible. Nevertheless, given the scope and selection criteria used in our study, we believe that our findings can be generalized.

10.4 Conclusion validity

There is a risk that our study can not be reproduced. To mitigate it, we provide a replication package containing the list of papers retained after each step in the filtering process described in Sect. 3.5, along with their quality ratings (see Sect. 3.5.4) and extracted data (see Sect. 3.7) when applicable. Any script used to help us conduct the study is also provided in the replication package.

11 Conclusion

This study provides a comprehensive systematic review of the literature on DC4ML and ML4DC, i.e., data cleaning with and for ML. Our study includes 101 papers published between 2016 and 2022 inclusively, from various academic databases. We covered 6 different types of data cleaning activities: feature cleaning, label cleaning, entity matching, outlier detection, imputation, and holistic data cleaning. Based on our review, we provide 24 future research directions and we highlight some exciting data-cleaning approaches that can be further extended. Our future research directions revolve around 6 core ideas: (1) to improve data augmentation techniques for data cleaning, (2) to create more public data cleaning datasets, (3) to provide better

tooling for cleaning datasets, (4) to study how LLMs can be used for data cleaning, (5) to explore holistic data cleaning approaches, and (6) to develop more interactive data cleaning approaches. We hope that this paper will serve as a solid foundation for future works on this important topic.

Acknowledgements This work is funded by the Fonds de Recherche du Quebec (FRQ), the Canadian Institute for Advanced Research (CIFAR), and the Natural Sciences and Engineering Research Council of Canada (NSERC). We would like to thank Dr. Hyacinth Ali for contributing to improving this SLR with his valuable comments.

Author Contributions Conceptualization: P-O.C., A.N., F.K.; Paper collection: P-O.C.; Paper selection: P-O.C., A.N.; Data Extraction: P-O.C. (74 papers), N.A. (17 papers), D.H. (6 papers), A.N. (2 papers); Writing - original draft preparation: P-O.C., N.A., D.H., A.N.; Writing - review and editing: A.N., P-O.C., F.K., D.H.; Funding acquisition: A.N., F.K.; Project administration: P-O.C.; Supervision: A.N., F.K.; Validation: A.N., P-O.C.; Software: P-O.C.; Resource: P-O.C.

Data availability All data generated or analyzed during this study are available in the GitHub repository to help reproduce our results (Côté et al. 2023).

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

- (2022) Common problems. <https://developers.google.com/machine-learning/gan/problems>
- (2023) <https://www.cnet.com/tech/chatgpt-can-pass-the-bar-exam-does-that-actually-matter/>
- Abedjan, Z., Chu, X., Deng, D., Fernandez, R.C., Ilyas, I.F., Ouzzani, M., Papotti, P., Stonebraker, M., Tang, N.: Detecting data errors: Where are we and what needs to be done? *Proc. VLDB Endow.* **9**(12), 993–1004 (2016)
- Abidin, N.Z., Ismail, A.R., Emran, N.A.: Performance analysis of machine learning algorithms for missing value imputation. *Int. J. Adv. Comput. Sci. Appl.* **9**(6), (2018)
- Abiteboul, S., Hull, R., Vianu, V.: *Foundations of Databases*, vol. 8. Addison-Wesley Reading, Delhi (1995)
- Adhikari, D., Jiang, W., Zhan, J., He, Z., Rawat, D.B., Aickelin, U., Khorshidi, H.A.: A comprehensive survey on imputation of missing data in internet of things. *ACM Comput. Surv.* **55**(7), 1–38 (2022)
- Aggarwal Charu, C., Reddy Chandan, K.: *Data clustering: algorithms and applications*, (2013)
- Agrawal, A., Chatterjee, R., Curino, C., Floratou, A., Gowdal, N., Interlandi, M., Jindal, A., Karanasos, K., Krishnan, S., Kroth, B., et al.: Cloudy with high chance of dbms: A 10-year prediction for enterprise-grade ml. (2019), arXiv preprint [arXiv:1909.00084](https://arxiv.org/abs/1909.00084)
- Akouemo, H.N., Povinelli, R.J.: Data improving in time series using ARX and ANN models. *IEEE Trans. Power Syst.* **32**(5), 3352–3359 (2017)
- Alimohammadi, H., Chen, S.N.: Performance evaluation of outlier detection techniques in production timeseries: A systematic review and meta-analysis. *Expert Syst. Appl.* **191**, 116371 (2022)
- Alsolai, H., Roper, M.: A systematic literature review of machine learning techniques for software maintainability prediction. *Inf. Softw. Technol.* **119**, 106214 (2020). <https://doi.org/10.1016/j.infsof.2019.106214>
- Dempster, A.P.: Upper and lower probabilities induced by a multivalued mapping. *Ann. Math. Stat.* **38**(2), 325–339 (1967)
- Araci, D.: Finbert: financial sentiment analysis with pre-trained language models. arXiv preprint [arXiv:1908.10063](https://arxiv.org/abs/1908.10063) (2019)
- Ataeyan, M., Daneshpour, N.: A novel data repairing approach based on constraints and ensemble learning. *Expert Syst. Appl.* **159**, 113511 (2020)

- Atkinson, G., Metsis, V.: Identifying label noise in time-series datasets. In: Adjunct Proceedings of the 2020 ACM International Joint Conference on Pervasive and Ubiquitous Computing and Proceedings of the 2020 ACM International Symposium on Wearable Computers, pp. 238–243 (2020)
- Atkinson, G., Metsis, V.: Tsar: a time series assisted relabeling tool for reducing label noise. In: The 14th Pervasive Technologies Related to Assistive Environments Conference, pp 203–209. (2021)
- Azeem, M.I., Palomba, F., Shi, L., Wang, Q.: Machine learning techniques for code smell detection: a systematic literature review and meta-analysis. *Inf. Softw. Technol.* **108**, 115–138 (2019). <https://doi.org/10.1016/j.infsof.2018.12.009>
- Bach, S.H., Broecheler, M., Huang, B., Getoor, L.: Hinge-loss Markov random fields and probabilistic soft logic. *J. Mach. Learn. Res.* **18**, 1–67 (2017)
- Badue, C., Guidolini, R., Carneiro, R.V., Azevedo, P., Cardoso, V.B., Forechi, A., Jesus, L., Berriel, R., Paixao, T.M., Mutz, F., et al.: Self-driving cars: a survey. *Expert Syst. Appl.* **165**, 113816 (2021)
- Bagherzadeh, P., Sadoghi Yazdi, H.: Label denoising based on Bayesian aggregation. *Int. J. Mach. Learn. Cybern.* **8**, 903–914 (2017)
- Bank, D., Koenigstein, N., Giryres, R.: Autoencoders. arXiv preprint [arXiv:2003.05991](https://arxiv.org/abs/2003.05991) (2020)
- Barlaug, N., Gulla, J.A.: Neural networks for entity matching: a survey. *ACM Trans. Knowl. Discov. Data (TKDD)* **15**(3), 1–37 (2021)
- Beltagy, I., Lo, K., Cohan, A.: Scibert: a pretrained language model for scientific text. arXiv preprint [arXiv:1903.10676](https://arxiv.org/abs/1903.10676) (2019)
- Ben-Gal, I.: Outlier detection in: data mining and knowledge discovery handbook: A complete guide for practitioners and researchers (2005)
- Bergstra, J., Yamins, D., Cox, D.D., et al.: Hyperopt: a python library for optimizing the hyperparameters of machine learning algorithms. In: Proceedings of the 12th Python in science conference, Citeseer, vol. 13, p. 20 (2013)
- Bernhardt, M., Castro, D.C., Tanno, R., Schwaighofer, A., Tezcan, K.C., Monteiro, M., Bannur, S., Lungren, M.P., Nori, A., Glocker, B., et al.: Active label cleaning for improved dataset quality under resource constraints. *Nat. Commun.* **13**(1), 1161 (2022)
- Berti-Equille, L.: Learn2clean: Optimizing the sequence of tasks for web data preparation. In: The World Wide Web Conference, pp. 2580–2586 (2019)
- Bhandari, K., Kumar, K., Sangal, A.L.: Data quality issues in software fault prediction: a systematic literature review. *Artif. Intelli. Rev.* **56**(8), 7839–7908 (2023)
- Bishop, C.M., Nasrabadi, N.M.: Pattern Recognition and Machine Learning, vol. 4. Springer, New York (2006)
- Bogatu, A., Paton, N.W., Douthwaite, M., Davie, S., Freitas, A.: Cost-effective variational active entity resolution. In: 2021 IEEE 37th International Conference on Data Engineering (ICDE), IEEE, pp. 1272–1283 (2021)
- Bojanowski, P., Grave, E., Joulin, A., Mikolov, T.: Enriching word vectors with subword information. *Trans. Assoc. Comput. Linguist.* **5**, 135–146 (2017)
- Bosu, M.F., MacDonell, S.G.: A taxonomy of data quality challenges in empirical software engineering. In: 2013 22nd Australian Software Engineering Conference, IEEE, pp. 97–106 (2013)
- Boukerche, A., Zheng, L., Alfandi, O.: Outlier detection: methods, models, and classification. *ACM Comput. Surv. (CSUR)* **53**(3), 1–37 (2020)
- Braick, H.B., Khomh, F.: On testing machine learning programs. *J. Syst. Softw.* **164**, 110542 (2020). <https://doi.org/10.1016/j.jss.2020.110542>
- Brunner, U., Stockinger, K.: Entity matching with transformer architectures-a step forward in data integration. In: 23rd International Conference on Extending Database Technology, Copenhagen, Open-Proceedings (2020)
- Carpenter, G.A., Grossberg, S., Rosen, D.B.: Fuzzy art: fast stable learning and categorization of analog patterns by an adaptive resonance system. *Neural Netw* **4**(6), 759–771 (1991)
- Cer, D., Yang, Y., Kong, Sy., Hua, N., Limtiaco, N., John, R.S., Constant, N., Guajardo-Cespedes, M., Yuan, S., Tar, C., et al.: Universal sentence encoder. arXiv preprint [arXiv:1803.11175](https://arxiv.org/abs/1803.11175) (2018)
- Chai, C., Wang, J., Luo, Y., Niu, Z., Li, G.: Data management for machine learning: a survey. *IEEE Trans. Knowl. Data Eng.* **35**(5), 4646–4667 (2022)
- Chasmai, M.E.: Cubetr: learning to solve the rubiks cube using transformers. arXiv preprint [arXiv:2111.06036](https://arxiv.org/abs/2111.06036) (2021)
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto HPdO, Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al.: Evaluating large language models trained on code (2021). arXiv preprint [arXiv:2107.03374](https://arxiv.org/abs/2107.03374)

- Chen, T., Kornblith, S., Norouzi, M., Hinton, G.: A simple framework for contrastive learning of visual representations. In: International conference on machine learning, PMLR, pp. 1597–1607. (2020)
- Cheng, K., Li, X., Xu, Y.E., Dong, X.L., Sun, Y.: Pge: Robust Product Graph Embedding Learning for Error Detection. <https://doi.org/10.48550/ARXIV.2202.09747>. [arXiv:2202.09747](https://arxiv.org/abs/2202.09747) (2022)
- Cholewiak, S.A., Ipeirotis, P., Silva, V., Kannawadi, A.: SCHOLARLY: Simple Access to Google Scholar Authors and Citation Using Python. <https://doi.org/10.5281/zenodo.5764801>, <https://github.com/scholarly-python-package/scholarly> (2021)
- Christophides, V., Efthymiou, V., Palpanas, T., Papadakis, G., Stefanidis, K.: An overview of end-to-end entity resolution for big data. *ACM Comput. Surv. (CSUR)* **53**(6), 1–42 (2020)
- Chu, X., Ilyas, I.F., Krishnan, S., Wang, J.: Data cleaning: Overview and emerging challenges. In: Proceedings of the 2016 International Conference on Management of Data, Association for Computing Machinery, New York, NY, USA, SIGMOD '16, pp. 2201–2206. <https://doi.org/10.1145/2882903.2912574> (2016a)
- Chu, X., Ilyas, I.F., Krishnan, S., Wang, J.: Data cleaning: overview and emerging challenges. In: Proceedings of the 2016 International Conference on Management of Data, pp. 2201–2206. (2016b)
- Côté, P.O., Nikanjam, A., Bouchoucha, R., Basta, I., Abidi, M., Khomh, F.: Quality Issues in Machine Learning Software Systems. *arXiv preprint [arXiv:2306.15007](https://arxiv.org/abs/2306.15007)* (2023)
- Croft, R., Xie, Y., Babar, M.A.: Data preparation for software vulnerability prediction: a systematic literature review. *IEEE Trans. Softw. Eng.* **49**(3), 1044–1063 (2022)
- Croft, R., Babar, M.A., Kholoosi, M.M.: Data quality for software vulnerability datasets. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE, pp. 121–133 (2023)
- Cubuk, E.D., Zoph, B., Mane, D., Vasudevan, V., Le, Q.V.: Autoaugment: learning augmentation strategies from data. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 113–123. (2019)
- Côté, P.O., Nikanjam, A., Ahmed, N., Humeniuk, D., Khomh, F.: The replication package. <https://github.com/poclecoq/SLR-datacleaning> (2023)
- Das, S., Doan, A., G C PS., Gokhale, C., Konda, P., Govind, Y., Paulsen, D.: The Magellan Data Repository. <https://sites.google.com/site/anhaidgroup/projects/data> (2016)
- Dempster, A.P., et al.: Upper and lower probabilities induced by a multivalued mapping. In: Classic Works of the Dempster-Shafer Theory of Belief Functions, pp. 57–72. Springer, Berlin (2008)
- Deng, D., Fernandez, R.C., Abedjan, Z., Wang, S., Stonebraker, M., Elmagarmid, A.K., Ilyas, I.F., Mad-den, S., Ouzzani, M., Tang, N.: The data civilizer system. In: Cidr, (2017)
- Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: Imagenet: A large-scale hierarchical image database. In: 2009 IEEE Conference on Computer Vision and Pattern Recognition, IEEE, pp. 248–255. (2009)
- Dolatshah, M., Teoh, M., Wang, J., Pei, J.: Cleaning crowdsourced labels using oracles for supervised learning. *PVLDB* **12**(4), 376–389 (2018)
- Domingues, R., Filippone, M., Michiardi, P., Zouaoui, J.: A comparative evaluation of outlier detection algorithms: experiments and analyses. *Pattern Recognit.* **74**, 406–421 (2018)
- Dong, X.L., Rekatsinas, T.: Data integration and machine learning: a natural synergy. In: Proceedings of the 2018 International Conference on Management of Data, pp. 1645–1650. (2018)
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al.: An image is worth 16x16 words: transformers for image recognition at scale (2020). *arXiv preprint [arXiv:2010.11929](https://arxiv.org/abs/2010.11929)*
- Ebraheem, M., Thirumuruganathan, S., Joty, S., Ouzzani, M., Tang, N.: Distributed representations of tuples for entity resolution. *Proc. VLDB Endow* **11**(11), 1454–1467 (2018)
- Ekambaram, R., Fefilatye, S., Shreve, M., Kramer, K., Hall, L.O., Goldgof, D.B., Kasturi, R.: Active cleaning of label noise. *Pattern Recognit.* **51**, 463–480 (2016)
- Felderer, M., Russo, B., Auer, F.: On testing data-intensive software systems. In: Security and Quality in Cyber-Physical Systems Engineering: With Forewords by Robert M Lee and Tom Gilb, pp. 129–148. (2019)
- Feldt, R., Magazinius, A.: Validity threats in empirical software engineering research-an initial survey. In: Seke, pp 374–379. (2010)
- Feng, W., Long, Y., Wang, S., Quan, Y.: A review of addressing class noise problems of remote sensing classification. *J. Syst. Eng. Electron.* **34**(1), 36–46 (2023). <https://doi.org/10.23919/JSEE.2023.000034>
- Filippone, M., Sanguinetti, G.: Information theoretic novelty detection. *Pattern Recognit.* **43**(3), 805–814 (2010)

- Flokas, L., Wu, W., Liu, Y., Wang, J., Verma, N., Wu, E.: Complaint-driven training data debugging at interactive speeds. In: Proceedings of the 2022 International Conference on Management of Data, pp 369–383. (2022)
- Foidl, H., Felderer, M.: Risk-based data validation in machine learning-based software systems. In: Proceedings of the 3rd ACM SIGSOFT International Workshop on Machine Learning Techniques for Software Quality Evaluation, pp. 13–18 (2019)
- Fox, T.L., Guynes, C.S., Prybutok, V.R., Windsor, J.: Maintaining quality in information systems. *J. Comput. Inf. Syst.* **40**(1), 76–80 (1999)
- Freund, Y., Seung, H.S., Shamir, E., Tishby, N.: Selective sampling using the query by committee algorithm. *Mach. Learn.* **28**(2–3), 133 (1997)
- Fu, C., Han, X., He, J., Sun, L.: Hierarchical matching network for heterogeneous entity resolution. In: Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence, pp. 3665–3671. (2021)
- Gal, Y.: Uncertainty in Deep Learning (2016)
- Gal, Y., Ghahramani, Z.: Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. <https://doi.org/10.48550/ARXIV.1506.02142>, [arXiv:1506.02142](https://arxiv.org/abs/1506.02142) (2015)
- Gauen, K., Dailey, R., Laiman, J., Zi, Y., Asokan, N., Lu, Y.H., Thiruvathukal, G.K., Shyu, M.L., Chen, S.C.: Comparison of visual datasets for machine learning. In: 2017 IEEE International Conference on Information Reuse and Integration (IRI), IEEE, pp. 346–355. (2017)
- Ge, C., Gao, Y., Miao, X., Yao, B., Wang, H.: A hybrid data cleaning framework using Markov logic networks. *IEEE Trans. Knowl. Data Eng.* **34**(5), 2048–2062 (2020)
- Gemp, I., Theocharous, G., Ghavamzadeh, M.: Automated Data Cleansing Through Meta-learning. In: Twenty-Ninth IAAI Conference (2017)
- Gezici, B., Tarhan, A.K.: Systematic literature review on software quality for AI-based software. *Empir. Softw. Eng.* **27**(3), 66 (2022)
- Gitnux, A.: Self driving cars safety statistics and trends in 2023 • gitnux. <https://blog.gitnux.com/self-driving-cars-safety-statistics/> (2023)
- Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press. <http://www.deeplearningbook.org> (2016)
- Goodfellow, I.J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative adversarial networks (2014). [arXiv:1406.2661](https://arxiv.org/abs/1406.2661)
- Gottapu, R.D., Dagli, C., Ali, B.: Entity resolution using convolutional neural network. *Procedia Comput. Sci.* **95**, 153–158 (2016)
- Grill, J.B., Strub, F., Altché, F., Tallec, C., Richemond, P., Buchatskaya, E., Doersch, C., Avila Pires, B., Guo, Z., Gheshlaghi Azar, M., Piot, B., kavukcuoglu, k., Munos, R., Valko, M.: Bootstrap your own latent—a new approach to self-supervised learning. In: Larochelle H., Ranzato M., Hadsell R., Balcan M., Lin H. (eds.) *Advances in Neural Information Processing Systems*, Curran Associates, Inc., vol. 33, pp. 21271–21284. <https://proceedings.neurips.cc/paper/2020/file/f3ada80d5c4ee70142b17b8192b2958e-Paper.pdf> (2020)
- Guan, H., Zhang, Y., Xian, M., Cheng, H.D., Tang, X.: Wenn for individualized cleaning in imbalanced data. In: 2016 23rd International Conference on Pattern Recognition (ICPR), IEEE, pp. 456–461. (2016)
- Guo, G., Adjero, D., Li, X.: Automated cleaning of identity label noise in a large-scale face dataset using a face image quality control (2018)
- Guo, Y., Bettaieb, S.: An investigation of quality issues in vulnerability detection datasets. In: 2023 IEEE European Symposium on Security and Privacy Workshops (EuroS &PW), IEEE, pp. 29–33. (2023)
- Guo, Z., Rekatsinas, T.: Learning functional dependencies with sparse regression. [arXiv:1905.01425](https://arxiv.org/abs/1905.01425) (2019)
- Han, B., Yao, Q., Yu, X., Niu, G., Xu, M., Hu, W., Tsang, I., Sugiyama, M.: Co-teaching: Robust training of deep neural networks with extremely noisy labels. <https://doi.org/10.48550/ARXIV.1804.06872>, [arXiv:1804.06872](https://arxiv.org/abs/1804.06872) (2018)
- Hara, S., Nitanda, A., Maehara, T.: Data cleansing for models trained with SGD. *Adv. Neural Inf. Process. Syst.* **32**, (2019)
- Hawkins, D.M.: Identification of Outliers, vol. 11. Springer (1980)
- He, X., Zhao, K., Chu, X.: Automl: a survey of the state-of-the-art. *Knowl. Based Syst.* **212**, 106622 (2021a)
- He, Y. et al.: Automatic detection of grammatical errors in English verbs based on rnn algorithm: auxiliary objectives for neural error detection models. *Comput. Intell. Neurosci.* (2021b)

- Heidari, A., McGrath, J., Ilyas, I.F., Rekatsinas, T.: Holodetect: few-shot learning for error detection. In: Proceedings of the 2019 International Conference on Management of Data, pp. 829–846 (2019)
- Hendrycks, D., Gimpel, K.: A baseline for detecting misclassified and out-of-distribution examples in neural networks. <https://doi.org/10.48550/ARXIV.1610.02136>, [arXiv:1610.02136](https://arxiv.org/abs/1610.02136) (2016)
- Hernández-García, A., König, P.: Data augmentation instead of explicit regularization. [arXiv preprint arXiv:1806.03852](https://arxiv.org/abs/1806.03852) (2018)
- Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Comput.* **9**(8), 1735–1780 (1997)
- Huang, J., Qu, L., Jia, R., Zhao, B.: O2u-net: A simple noisy label detection approach for deep neural networks. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 3326–3334. (2019)
- Huang, J., Hu, W., Bao, Z., Chen, Q., Qu, Y.: Deep entity matching with adversarial active learning. *VLDB J.* **32**(1), 229–255 (2023)
- Huang, Z., Li, X., Deng, L., Wei, K., Sui, Y.: Mislabeled samples adjustment based on self-paced learning framework. In: 2021 7th International Conference on Computer and Communications (ICCC), IEEE, pp. 1659–1659. (2021)
- Hurakadli, V., Kulkarni, S., Patil, U., Tabib, R., Mudengudi, U.: Deep learning based radial blur estimation and image enhancement. In: 2019 IEEE International Conference on Electronics, pp. 1–5. IEEE, Computing and Communication Technologies (CONECCT) (2019)
- Hwang, P., Kim, Y.: Data cleaning of sound data with label noise using self organizing map. In: 2022 16th International Conference on Ubiquitous Information Management and Communication (IMCOM), pp 1–5. <https://doi.org/10.1109/IMCOM53663.2022.9721724> (2022)
- Ilyas, I., Chu, X.: Data Cleaning. Association for Computing Machinery and Morgan & Claypool Publishers. <https://books.google.ca/books?id=RxieDwAAQBAJ> (2019).
- Ilyas, I.F., Rekatsinas, T.: Machine learning and data cleaning: Which serves the other? *J. Data Inf. Qual.* **14**(3), 1–11 (2022). <https://doi.org/10.1145/3506712>
- Indyk, P., Motwani, R.: Approximate nearest neighbors: Towards removing the curse of dimensionality. In: Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing, Association for Computing Machinery, New York, NY, USA, STOC '98, pp. 604–613. <https://doi.org/10.1145/276698.276876> (1998)
- Jiang, W., Ge, Y., Cheng, H., Chen, M., Feng, S., Wang, C.: Read: aggregating reconstruction error into out-of-distribution detection. *Proc. AAAI Conf. Artif. Intell.* **37**, 14910–14918 (2023)
- Jin, D., Sisman, B., Wei, H., Dong, X.L., Koutra, D.: Deep transfer learning for multi-source entity linkage via domain adaptation. [arXiv preprint arXiv:2110.14509](https://arxiv.org/abs/2110.14509) (2021)
- Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. *IEEE Trans. Big Data* **7**(3), 535–547 (2019)
- Johnson, J.M., Khoshgoftaar, T.M.: A survey on classifying big data with label noise. *ACM J. Data Inf. Qual.* **14**(4), 1–43 (2022)
- Kang, Z., Catal, C., Tekinerdogan, B.: Machine learning applications in production lines: a systematic literature review. *Comput. Ind. Eng.* **149**, 106773 (2020). <https://doi.org/10.1016/j.cie.2020.106773>
- Karimi, D., Dou, H., Warfield, S.K., Gholipour, A.: Deep learning with noisy labels: exploring techniques and remedies in medical image analysis. *Med. Image Anal.* **65**, 101759 (2020)
- Karlaš, B., Li, P., Wu, R., Gürel, N.M., Chu, X., Wu, W., Zhang, C.: Nearest neighbor classifiers over incomplete information: from certain answers to certain predictions. [arXiv preprint arXiv:2005.05117](https://arxiv.org/abs/2005.05117) (2020)
- Kasai, J., Qian, K., Gurajada, S., Li, Y., Popa, L.: Low-resource deep entity resolution with transfer and active learning. [arXiv preprint arXiv:1906.08042](https://arxiv.org/abs/1906.08042) (2019)
- Ke, X., Bai, J., Wen, L., Cao, B.: Multi-index dialogue data cleaning model. In: 2019 IEEE 8th Joint International Information Technology and Artificial Intelligence Conference (ITAIC), IEEE, pp. 672–676. (2019)
- Kim, J., Scott, C.D.: Robust kernel density estimation. *J. Mach. Learn. Res.* **13**(1), 2529–2565 (2012)
- Kitchenham, B.: Procedures for performing systematic reviews. *Keele UK Keele Univ.* **33**(2004), 1–26 (2004)
- Klie, J.C., Webber, B., Gurevych, I.: Annotation error detection: Analyzing the past and present for a more coherent future. *Comput. Linguist.* pp. 1–42 (2022)
- Knill, K.M., Gales, M.J., Manakul, P., Caines, A.: Automatic grammatical error detection of non-native spoken learner english. In: ICASSP 2019–2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, pp. 8127–8131. (2019)

- Koh, P.W., Liang, P.: Understanding black-box predictions via influence functions. In: International Conference on Machine Learning, PMLR, pp. 1885–1894 (2017)
- Köhler, J.M., Autenrieth, M., Beluch, W.H.: Uncertainty based detection and relabeling of noisy image labels. In: CVPR Workshops, pp. 33–37. (2019)
- Kriegel, H.P., Kröger, P., Schubert, E., Zimek, A.: Outlier detection in axis-parallel subspaces of high dimensional data. In: Advances in Knowledge Discovery and Data Mining: 13th Pacific-Asia Conference, PAKDD 2009 Bangkok, Thailand, 2009 Proceedings 13, Springer, pp. 831–838. (2009)
- Krishnan, S., Wu, E.: Alphaclean: Automatic generation of data cleaning pipelines. arXiv preprint [arXiv:1904.11827](https://arxiv.org/abs/1904.11827) (2019)
- Krishnan, S., Wang, J., Wu, E., Franklin, M.J., Goldberg, K.: Activeclean: interactive data cleaning for statistical modeling. *Proc. VLDB Endow.* **9**(12), 948–959 (2016)
- Krishnan, S., Franklin, M.J., Goldberg, K., Wu, E.: Boostclean: Automated error detection and repair for machine learning. arXiv preprint [arXiv:1711.01299](https://arxiv.org/abs/1711.01299) (2017)
- Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. *Commun. ACM* **60**(6), 84–90 (2017)
- Lakshminarayan, K., Harp, S.A., Samad, T.: Imputation of missing data in industrial databases. *Appl. Intell.* **11**(3), 259–275 (1999)
- Lakshminarayanan, B., Pritzel, A., Blundell, C.: Simple and scalable predictive uncertainty estimation using deep ensembles. <https://doi.org/10.48550/ARXIV.1612.01474>, [arXiv:1612.01474](https://arxiv.org/abs/1612.01474) (2016)
- Lattar, H., Salem, A.B., Ghezala, H.H.B.: Does data cleaning improve heart disease prediction? *Proc. Comput. Sci.* **176**, 1131–1140 (2020)
- Laure, B.E., Angela, B., Tova, M.: Machine learning to data management: A round trip. In: 2018 IEEE 34th International Conference on Data Engineering (ICDE), IEEE, pp. 1735–1738. (2018)
- Lee, K.H., He, X., Zhang, L., Yang, L.: Cleannet: Transfer learning for scalable image classifier training with label noise. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 5447–5456. (2018)
- Lew, A., Agrawal, M., Sontag, D., Mansinghka, V.: Pclean: Bayesian data cleaning at scale with domain-specific probabilistic programming. In: International Conference on Artificial Intelligence and Statistics, PMLR, pp. 1927–1935. (2021)
- Li, B., Wang, W., Sun, Y., Zhang, L., Ali, M.A., Wang, Y.: Grapher: token-centric entity resolution with graph convolutional neural networks. *Proc. AAAI Conf. Artif. Intell.* **34**, 8172–8179 (2020)
- Li, P., Rao, X., Blase, J., Zhang, Y., Chu, X., Zhang, C.: Cleanml: A benchmark for joint data cleaning and machine learning [experiments and analysis], p 75. arXiv preprint [arXiv:1904.09483](https://arxiv.org/abs/1904.09483) (2019)
- Li, Y., Li, J., Suhara, Y., Doan, A., Tan, W.C.: Deep entity matching with pre-trained language models. arXiv preprint [arXiv:2004.00584](https://arxiv.org/abs/2004.00584) (2020b)
- Li, Z., Du, W., Rao, N.: Research on error label screening method based on convolutional neural network. In: 2021 IEEE 6th International Conference on Signal and Image Processing (ICSIP), IEEE, pp 1020–1024. (2021)
- Liang, Q., Sun, Z., Zhu, Q., Hu, J., Zhao, Y., Zhang, L.: Cupcleaner: A data cleaning approach for comment updating. arXiv preprint [arXiv:2308.06898](https://arxiv.org/abs/2308.06898) (2023)
- Liebchen, G., Shepperd, M.: Data sets and data quality in software engineering: Eight years on. In: Proceedings of the The 12th International Conference on Predictive Models and Data Analytics in Software Engineering, Association for Computing Machinery, New York, NY, USA, PROMISE 2016. <https://doi.org/10.1145/2972958.2972967> (2016)
- Liebchen, G.A., Shepperd, M.: Data sets and data quality in software engineering. In: Proceedings of the 4th International Workshop on Predictor Models in Software Engineering, pp 39–44. (2008)
- Lim, S., Kim, I., Kim, T., Kim, C., Kim, S.: Fast autoaugment. *Adv. Neural Inf. Process. Syst.* **32**, (2019)
- Lin, W.C., Tsai, C.F.: Missing value imputation: a review and analysis of the literature (2006–2017). *Artif. Intell. Rev.* **53**, 1487–1509 (2020)
- Liu, D., Meng, Y., Wang, L.: Data cleaning of irrelevant images based on transfer learning. In: 2020 International Conference on Intelligent Computing, Automation and Systems (ICICAS), pp. 450–456. IEEE, (2020)
- Liu, F.T., Ting, K.M., Zhou, Z.H.: Isolation forest. In: 2008 Eighth IEEE International Conference on Data Mining, IEEE, pp. 413–422 (2008)
- Liu, Y., Li, Z., Zhou, C., Jiang, Y., Sun, J., Wang, M., He, X.: Generative adversarial active learning for unsupervised outlier detection. *IEEE Trans. Knowl. Data Eng.* **32**(8), 1517–1528 (2019)
- Liu, Z., Zhou, Z., Rekatsinas, T.: Picket: guarding against corrupted data in tabular data during learning and inference. *VLDB J.* pp. 1–29 (2022)

- Mahdavi, M., Abedjan, Z.: Baran: effective error correction via a unified context representation and transfer learning. *Proc. VLDB Endow.* **13**(12), 1948–1961 (2020)
- Mahdavi, M., Abedjan, Z.: Semi-supervised data cleaning with raha and baran. In: CIDR, (2021)
- Mahdavi, M., Abedjan, Z., Castro Fernandez, R., Madden, S., Ouzzani, M., Stonebraker, M., Tang, N.: Raha: A configuration-free error detection system. In: *Proceedings of the 2019 International Conference on Management of Data*, pp. 865–882. (2019)
- Marsland, S., Shapiro, J., Nehmzow, U.: A self-organising network that grows when required. *Neural Netw.* **15**(8–9), 1041–1058 (2002)
- Martínez-Fernández, S., Bogner, J., Franch, X., Oriol, M., Siebert, J., Trendowicz, A., Vollmer, A.M., Wagner, S.: Software engineering for AI-based systems: a survey. *ACM Trans. Softw. Eng. Methodol.* **31**(2), 1–59 (2022). <https://doi.org/10.1145/3487043>
- Mauritz, R., Nijweide, F., Goseling, J., van Keulen, M.: A probabilistic database approach to autoencoder-based data cleaning. *arXiv preprint arXiv:2106.09764* (2021)
- Mayfield, C., Neville, J., Prabhakar, S.: Eracer: a database approach for statistical inference and data cleaning. In: *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pp 75–86. (2010)
- Mazumder, M., Banbury, C., Yao, X., Karlaš, B., Rojas, W.G., Diamos, S., Diamos, G., He, L., Parrish, A., Kirk, H.R., et al.: Dataperf: Benchmarks for data-centric AI development. *arXiv preprint arXiv:2207.10062* (2022)
- Meduri, V.V., Popa, L., Sen, P., Sarwat, M.: A comprehensive benchmark framework for active learning methods in entity matching. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pp. 1133–1147. (2020)
- Miao, Z., Li, Y., Wang, X.: Rotom: A meta-learned data augmentation framework for entity matching, data cleaning, text classification, and beyond. In: *Proceedings of the 2021 International Conference on Management of Data*, pp. 1303–1316. (2021)
- Motulsky, H.J., Brown, R.E.: Detecting outliers when fitting data with nonlinear regression—a new method based on robust nonlinear regression and the false discovery rate. *BMC Bioinform.* **7**(1), 1–20 (2006)
- Mudgal, S., Li, H., Rekatsinas, T., Doan, A., Park, Y., Krishnan, G., Deep, R., Arcaute, E., Raghavendra, V.: Deep learning for entity matching: A design space exploration. In: *Proceedings of the 2018 International Conference on Management of Data*, pp 19–34, (2018)
- Müller, H., Castelo, S., Qazi, M., Freire, J.: From papers to practice: the openclean open-source data cleaning library. *Proc. VLDB Endow* **14**(12), 2763–2766 (2021)
- Narayan, A., Chami, I., Orr, L., Ré, C.: Can foundation models wrangle your data? (2022). *arXiv preprint arXiv:2205.09911*
- Nashaat, M., Ghosh, A., Miller, J., Quader, S.: Tabreformer: unsupervised representation learning for erroneous data detection. *ACM/IMS Trans. Data Sci.* **2**(3), 1–29 (2021)
- Nassif, A.B., Talib, M.A., Nasir, Q., Dakalbab, F.M.: Machine learning for anomaly detection: a systematic review. *IEEE Access* **9**, 78658–78700 (2021)
- Neutatz, F., Mahdavi, M., Abedjan, Z.: Ed2: two-stage active learning for error detection—technical report. *arXiv preprint arXiv:1908.06309* (2019)
- Neutatz, F., Chen, B., Abedjan, Z., Wu, E.: From cleaning before ml to cleaning for ml. *IEEE Data Eng. Bull.* **44**(1), 24–41 (2021)
- Ng, A.: A chat with andrew on mlps: from model-centric to data-centric AI. https://www.youtube.com/watch?v=06-AZXmwHjo&ab_channel=DeepLearningAI (2021)
- Ng, A., He, L., Laird, D.: Data-centric AI competition. <https://https-deeplearning-ai.github.io/data-centric-comp/> (2021)
- Nie, H., Han, X., He, B., Sun, L., Chen, B., Zhang, W., Wu, S., Kong, H.: Deep sequence-to-sequence entity matching for heterogeneous entity resolution. In: *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, pp. 629–638. (2019)
- Northcutt, C.G., Jiang, L., Chuang, I.L.: Confident learning: Estimating uncertainty in dataset labels. <https://doi.org/10.48550/ARXIV.1911.00068>, *arXiv:1911.00068* (2019)
- Oliveira, P.H., Kaster, D.S., Ilyas, I.F., et al.: Batchwise probabilistic incremental data cleaning. *arXiv preprint arXiv:2011.04730* (2020)
- OpenAI (2023) <https://openai.com/research/gpt-4>
- Pang, G., Shen, C., Cao, L., Hengel, A.V.D.: Deep learning for anomaly detection: a review. *ACM Comput. Surv. (CSUR)* **54**(2), 1–38 (2021)

- Papastefanopoulos, V., Linardatos, P., Kotsiantis, S.: Unsupervised outlier detection: a meta-learning algorithm based on feature selection. *Electronics* **10**(18), 2236 (2021)
- Patel, H., Gupta, N., Panwar, N., Sharma Mittal, R., Mehta, S., Guttula, S., Mujumdar, S., Afzal, S., Bedathur, S., Munigala, V.: Automatic assessment of quality of your data for AI. In: Proceedings of the 5th Joint International Conference on Data Science & Management of Data (9th ACM IKDD CODS and 27th COMAD), Association for Computing Machinery, New York, NY, USA, CODS-COMAD '22, pp. 354–357. (2022). <https://doi.org/10.1145/3493700.3493774>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: machine learning in Python. *J. Mach. Learn. Res.* **12**, 2825–2830 (2011)
- Pennington, J., Socher, R., Manning, C.D.: Glove: Global vectors for word representation. In: Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP), pp. 1532–1543. (2014)
- Pham, M., Knoblock, C.A., Chen, M., Vu, B., Pujara, J.: Spade: a semi-supervised probabilistic approach for detecting errors in tables. In: *IJCAI*, pp 3543–3551. (2021)
- Pise, N.N., Kulkarni, P.: A survey of semi-supervised learning methods. In: 2008 International Conference on Computational Intelligence and Security, IEEE, vol. 2, pp. 30–34. (2008)
- Pit-Claudel, C., Mariet, Z., Harding, R., Madden, S.: Outlier detection in heterogeneous datasets using automatic tuple expansion. Tech. rep., MIT—Computer Science and Artificial Intelligence Laboratory (MIT-CSAIL-TR-2016-002). (2016)
- Ponzio, F., Macii, E., Ficarra, E., Di Cataldo, S.: W2wnet: a two-module probabilistic convolutional neural network with embedded data cleansing functionality. *arXiv preprint arXiv:2103.13107* (2021)
- Pouyanfar, S., Sadiq, S., Yan, Y., Tian, H., Tao, Y., Reyes, M.P., Shyu, M.L., Chen, S.C., Iyengar, S.S.: A survey on deep learning: algorithms, techniques, and applications. *ACM Comput. Surv. (CSUR)* **51**(5), 1–36 (2018)
- Press, G.: Cleaning big data: Most time-consuming, least enjoyable data science task, survey says. <https://www.forbes.com/sites/gilpress/2016/03/23/data-preparation-most-time-consuming-least-enjoyable-data-science-task-survey-says/?sh=4c577cb46f63> (2022)
- Qian, K., Popa, L., Sen, P.: Active learning for large-scale entity resolution. In: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, pp. 1379–1388. (2017)
- Rahm, E., Do, H.H., et al.: Data cleaning: problems and current approaches. *IEEE Data Eng. Bull.* **23**(4), 3–13 (2000)
- Razavi-Far, R., Cheng, B., Saif, M., Ahmadi, M.: Similarity-learning information-fusion schemes for missing data imputation. *Knowl. Based Syst.* **187**, 104805 (2020)
- Rehbein, I., Ruppenhofer, J.: Detecting annotation noise in automatically labelled data. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Vol. 1: Long Papers), pp. 1160–1170. (2017)
- Rei, M., Yannakoudakis, H.: Compositional sequence labeling models for error detection in learner writing. In: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Vol. 1: Long Papers), Association for Computational Linguistics, Berlin, Germany, pp. 1181–1191. <https://doi.org/10.18653/v1/P16-1112>, <https://aclanthology.org/P16-1112> (2016)
- Rekatsinas, T., Chu, X., Ilyas, I.F., Ré, C.: Holoclean: Holistic data repairs with probabilistic inference. *arXiv preprint arXiv:1702.00820* (2017)
- Ren, M., Zeng, W., Yang, B., Urtasun, R.: Learning to reweight examples for robust deep learning. In: International Conference on Machine Learning, PMLR, pp. 4334–4343. (2018)
- Roh, Y., Heo, G., Whang, S.E.: A survey on data collection for machine learning: a big data-AI integration perspective. *IEEE Trans. Knowl. Data Eng.* **33**(4), 1328–1347 (2019)
- Rosner, B.: Percentage points for a generalized esd many-outlier procedure. *Technometrics* **25**(2), 165–172 (1983)
- Rottmann, M., Reese, M.: Automated detection of label errors in semantic segmentation datasets via deep learning and uncertainty quantification. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, pp. 3214–3223. (2023)
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., et al.: Imagenet large scale visual recognition challenge. *Int. J. Comput. Vis.* **115**, 211–252 (2015)
- Salekshahrezaee, Z., Leevy, J.L., Khoshgoftaar, T.M.: A reconstruction error-based framework for label noise detection. *J. Big Data* **8**, 1–16 (2021)

- Sambasivan, N., Kapania, S., Highfill, H., Akrong, D., Paritosh, P., Aroyo, L.M.: “Everyone wants to do the model work, not the data work”: data cascades in high-stakes AI. In: proceedings of the 2021 CHI Conference on Human Factors in Computing Systems, pp. 1–15. (2021)
- Santos, E.A., Campbell, J.C., Hindle, A., Amaral, J.N.: Finding and correcting syntax errors using recurrent neural networks. *PeerJ PrePrints* **5**, e3123v1 (2017)
- Sarker, I.H.: Deep learning: a comprehensive overview on techniques, taxonomy, applications and research directions. *SN Comput. Sci.* **2**(6), 1–20 (2021)
- Schölkopf, B., Williamson, R.C., Smola, A., Shawe-Taylor, J., Platt, J.: Support vector method for novelty detection. *Adv. Neural Inf. Process. Syst.* **12**, (1999)
- Shi, J., Wu, J.: Distilling effective supervision for robust medical image segmentation with noisy labels. In: Medical Image Computing and Computer Assisted Intervention–MICCAI 2021: 24th International Conference, Strasbourg, France, 2021, Proceedings, Part I 24, Springer, pp. 668–677. (2021)
- Shi, L., Mu, F., Chen, X., Wang, S., Wang, J., Yang, Y., Li, G., Xia, X., Wang, Q.: Are we building on the rock? On the importance of data preprocessing for code summarization. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 107–119. (2022)
- Silva-Ramírez, E.L., Cabrera-Sánchez, J.F.: Co-active neuro-fuzzy inference system model as single imputation approach for non-monotone pattern of missing data. *Neural Comput. Appl.* **33**, 8981–9004 (2021)
- Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014)
- Smyth, L.: Training-Valuenet: A New Approach for Label Cleaning on Weakly-Supervised Datasets. University of Exeter, (2020)
- Song, H., Kim, M., Park, D., Shin, Y., Lee, J.G.: Learning from noisy labels with deep neural networks: a survey. *IEEE Trans. Neural Netw. Learn. Syst.* **34**(11), 8135–8153 (2023). <https://doi.org/10.1109/TNNLS.2022.3152527>
- Spithourakis, G.P., Augenstein, I., Riedel, S.: Numerically grounded language models for semantic error correction. *arXiv preprint arXiv:1608.04147* (2016)
- Studer, S., Bui, T.B., Drescher, C., Hanuschkin, A., Winkler, L., Peters, S., Müller, K.R.: Towards crispml (q): a machine learning process model with quality assurance methodology. *Mach. Learn. Knowl. Extr.* **3**(2), 392–413 (2021)
- Su, J., Gao, X., Qin, Y., Guo, S.: Correcting corrupted labels using mode dropping of acgan. In: 2021 15th International Symposium on Medical Information and Communication Technology (ISMICT), IEEE, pp. 98–103. (2021)
- Surameery, N.M.S., Shakor, M.Y.: Use chat gpt to solve programming bugs. *Int. J. Inf. Technol. Comput. Eng. (IJITC)* **3**(01), 17–22 (2023)
- Suzuki, K., Kobayashi, Y., Narihira, T.: Data cleansing for deep neural networks with storage-efficient approximation of influence functions. *arXiv preprint arXiv:2103.11807* (2021)
- Tae, K.H., Roh, Y., Oh, Y.H., Kim, H., Whang, S.E.: Data cleaning for accurate, fair, and robust models: A big data-AI integration approach. In: Proceedings of the 3rd International Workshop on Data Management for End-to-End Machine Learning, pp. 1–4. (2019)
- Tambon, F., Laberge, G., An, L., Nikanjam, A., Mindom, P.S.N., Pequignot, Y., Khomh, F., Antoniol, G., Merlo, E., Laviolette, F.: How to certify machine learning based safety-critical systems? A systematic literature review. *Autom. Softw. Eng.* **29**(2), 1–74 (2022)
- Tang, N., Fan, J., Li, F., Tu, J., Du, X., Li, G., Madden, S., Ouzzani, M.: Relational pretrained transformers towards democratizing data preparation [vision]. *arXiv preprint arXiv:2012.02469* (2020)
- Tawfik, N.S., Spruit, M.R.: Evaluating sentence representations for biomedical text: methods and experimental results. *J. Biomed. Inform.* **104**, 103396 (2020)
- Team, S.: Data-centric AI for the enterprise (2024). <https://snorkel.ai/#>
- Terrades, O.R., Berenguel, A., Gil, D.: A flexible outlier detector based on a topology given by graph communities. *Big Data Res.* **29**, 100332 (2022)
- Teso, S., Bontempelli, A., Giunchiglia, F., Passerini, A.: Interactive label cleaning with example-based explanations. *Adv. Neural Inf. Process. Syst.* **34**, 12966–12977 (2021)
- Tfwala, S.S., Wang, Y.M., Lin, Y.C., et al.: Prediction of missing flow records using multilayer perceptron and coactive neurofuzzy inference system. *Sci. World J.* (2013)
- Thekumparampil, K.K., Khetan, A., Lin, Z., Oh, S.: Robustness of conditional gans to noisy labels. *Adv. Neural Inf. Process. Syst.* **31**, (2018)

- Thirumuruganathan, S., Tang, N., Ouzzani, M., Doan, A.: Data curation with deep learning. In: EDBT, pp. 277–286. (2020)
- Tonolini, F., Moreno, P.G., Damianou, A., Murray-Smith, R.: Tomographic auto-encoder: unsupervised bayesian recovery of corrupted data. arXiv preprint [arXiv:2006.16938](https://arxiv.org/abs/2006.16938) (2020)
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Adv. Neural Inf. Process. Syst.* **30**, (2017a)
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. *CoRR abs/1706.03762*, [arXiv:1706.03762](https://arxiv.org/abs/1706.03762) (2017b)
- Veit, A., Alldrin, N., Chechik, G., Krasin, I., Gupta, A., Belongie, S.: Learning from noisy large-scale datasets with minimal supervision. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 839–847. (2017)
- Visengeriyeva, L., Abedjan, Z.: Metadata-driven error detection. In: *Proceedings of the 30th International Conference on Scientific and Statistical Database Management*, pp. 1–12. (2018)
- Visengeriyeva, L., Akbik, A., Kaul, M., Rabl, T., Markl, V.: Improving data quality by leveraging statistical relational learning. In: *ICIQ*, pp. 220–236. (2016)
- Wang, H., Bah, M.J., Hammad, M.: Progress in outlier detection techniques: a survey. *IEEE Access* **7**, 107964–108000 (2019). <https://doi.org/10.1109/ACCESS.2019.2932769>
- Wang, Q., Tan, Y.: Grammatical error detection with self attention by pairwise training. In: *2020 International Joint Conference on Neural Networks (IJCNN)*, IEEE, pp. 1–7. (2020)
- Wang, R., Li, Y., Wang, J.: Sudowoodo: Contrastive self-supervised learning for multi-purpose data integration and preparation. arXiv preprint [arXiv:2207.04122](https://arxiv.org/abs/2207.04122) (2022)
- Wang, X., Wang, C.: Time series data cleaning: a survey. *IEEE Access* **8**, 1866–1881 (2019)
- Wang, Z., Sisman, B., Wei, H., Dong, X.L., Ji, S.: Cordel: a contrastive deep learning approach for entity linkage. In: *2020 IEEE International Conference on Data Mining (ICDM)*, IEEE, pp. 1322–1327. (2020)
- Wei, J., Zou, K.: Eda: Easy data augmentation techniques for boosting performance on text classification tasks. arXiv preprint [arXiv:1901.11196](https://arxiv.org/abs/1901.11196) (2019)
- Whang, S.E., Roh, Y., Song, H., Lee, J.G.: Data collection and quality challenges in deep learning: a data-centric AI perspective. arXiv preprint [arXiv:2112.06409](https://arxiv.org/abs/2112.06409) (2021)
- Whang, S.E., Roh, Y., Song, H., Lee, J.G.: Data collection and quality challenges in deep learning: a data-centric AI perspective. *VLDB J.* **32**(4), 791–813 (2023)
- White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., Schmidt, D.C.: A prompt pattern catalog to enhance prompt engineering with chatgpt. arXiv preprint [arXiv:2302.11382](https://arxiv.org/abs/2302.11382) (2023)
- Wikipedia (2023a) https://en.wikipedia.org/wiki/Machine_learning
- Wikipedia (2023b) [https://en.wikipedia.org/wiki/Imputation_\(statistics\)](https://en.wikipedia.org/wiki/Imputation_(statistics))
- Wikipedia (2023c) Active learning (machine learning). [https://en.wikipedia.org/wiki/Active_learning_\(machine_learning\)](https://en.wikipedia.org/wiki/Active_learning_(machine_learning))
- Wikipedia (2023d) Boosting (machine learning). [https://en.wikipedia.org/wiki/Boosting_\(machine_learning\)](https://en.wikipedia.org/wiki/Boosting_(machine_learning))
- Wikipedia (2023e) Transfer learning. https://en.wikipedia.org/wiki/Transfer_learning
- Wohlin, C.: Guidelines for snowballing in systematic literature studies and a replication in software engineering. In: *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, pp. 1–10. (2014)
- Wong, W.E., Gao, R., Li, Y., Abreu, R., Wotawa, F.: A survey on software fault localization. *IEEE Trans. Softw. Eng.* **42**(8), 707–740 (2016). <https://doi.org/10.1109/TSE.2016.2521368>
- Wu, R., Chaba, S., Sawlani, S., Chu, X., Thirumuruganathan, S.: Zeroer: Entity resolution using zero labeled examples. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pp. 1149–1164. (2020)
- Wu, Y., Weimer, J., Davidson, S.B.: Chef: a cheap and fast pipeline for iteratively cleaning label uncertainties (technical report). arXiv preprint [arXiv:2107.08588](https://arxiv.org/abs/2107.08588) (2021)
- Xiang, S., Ye, X., Xia, J., Wu, J., Chen, Y., Liu, S.: Interactive correction of mislabeled training data. In: *2019 IEEE Conference on Visual Analytics Science and Technology (VAST)*, IEEE, pp 57–68. (2019)
- Yu, Q., Aizawa, K.: Unknown class label cleaning for learning with open-set noisy labels. In: *2020 IEEE International Conference on Image Processing (ICIP)*, IEEE, pp 1731–1735. (2020)
- Zha, D., Bhat, Z.P., Lai, K.H., Yang, F., Jiang, Z., Zhong, S., Hu, X.: Data-centric artificial intelligence: a survey. arXiv preprint [arXiv:2303.10158](https://arxiv.org/abs/2303.10158) (2023)

- Zhang, A., Song, S., Wang, J., Yu, P.S.: Time series data cleaning: From anomaly detection to anomaly repairing (technical report). arXiv preprint [arXiv:2003.12396](https://arxiv.org/abs/2003.12396) (2020a)
- Zhang, H., Cisse, M., Dauphin, Y.N., Lopez-Paz, D.: mixup: Beyond empirical risk minimization. arXiv preprint [arXiv:1710.09412](https://arxiv.org/abs/1710.09412)(2017).
- Zhang, Q., Fang, C., Ma, Y., Sun, W., Chen, Z.: A survey of learning-based automated program repair. *ACM Trans. Softw. Eng. Methodol.* **33**(2), 1–69 (2023). <https://doi.org/10.1145/3631974>
- Zhang, W., Tan, X.: Combining outlier detection and reconstruction error minimization for label noise reduction. In: 2019 IEEE International Conference on Big Data and Smart Computing (BigComp), IEEE, pp. 1–4. (2019)
- Zhang, W., Wang, D., Tan, X.: Data cleaning and classification in the presence of label noise with class-specific autoencoder. In: International Symposium on Neural Networks, Springer, pp. 256–264. (2018a)
- Zhang, W., Wei, H., Sisman, B., Dong, X.L., Faloutsos, C., Page, D.: Autoblock: A hands-off blocking framework for entity matching. In: Proceedings of the 13th International Conference on Web Search and Data Mining, pp. 744–752. (2020b)
- Zhang, X., Ji, Y., Nguyen, C., Wang, T.: Deepclean: data cleaning via question asking. In: 2018 IEEE 5th International Conference on Data Science and Advanced Analytics (DSAA), IEEE, pp. 283–292. (2018b)
- Zhang, X., Zhu, X., Wright, S.: Training set debugging using trusted items. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, (2018c)
- Zhang, Y., Zheng, S., Dalirrooyfard, M., Wu, P., Schneider, A., Raj, A., Nevmyvaka, Y., Chen, C.: Learning to abstain from uninformative data. arXiv preprint [arXiv:2309.14240](https://arxiv.org/abs/2309.14240) (2023b)
- Zhao, C., He, Y.: Auto-em: end-to-end fuzzy entity-matching using pre-trained deep models and transfer learning. In: The World Wide Web Conference, pp. 2413–2424. (2019)
- Zhao, Z., Liu, H.: Spectral feature selection for supervised and unsupervised learning. In: Proceedings of the 24th International Conference on Machine Learning, pp. 1151–1157. (2007)
- Zhou, X., Jin, Y., Zhang, H., Li, S., Huang, X.: A map of threats to validity of systematic literature reviews in software engineering. In: 2016 23rd Asia-Pacific Software Engineering Conference (APSEC), IEEE, pp. 153–160. (2016)
- Zhou, X., Liu, X., Wang, C., Zhai, D., Jiang, J., Ji, X.: Learning with noisy labels via sparse regularization. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 72–81. (2021)
- Zhu, X., Ghahramani, Z.: Learning from Labeled and Unlabeled Data with Label Propagation. ProQuest Number: INFORMATION TO ALL USERS (2002)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Pierre-Olivier Côté¹ · Amin Nikanjam¹ · Nafisa Ahmed¹ · Dmytro Humeniuk¹ · Foutse Khomh¹

✉ Pierre-Olivier Côté
pierre-olivier.cote@polymtl.ca

Amin Nikanjam
amin.nikanjam@polymtl.ca

Nafisa Ahmed
nafisa.abdelmutalab-ali-ahmed@polymtl.ca

Dmytro Humeniuk
dmytro.humeniuk@polymtl.ca

Foutse Khomh
foutse.khomh@polymtl.ca

¹ Polytechnique Montréal, Québec, Canada